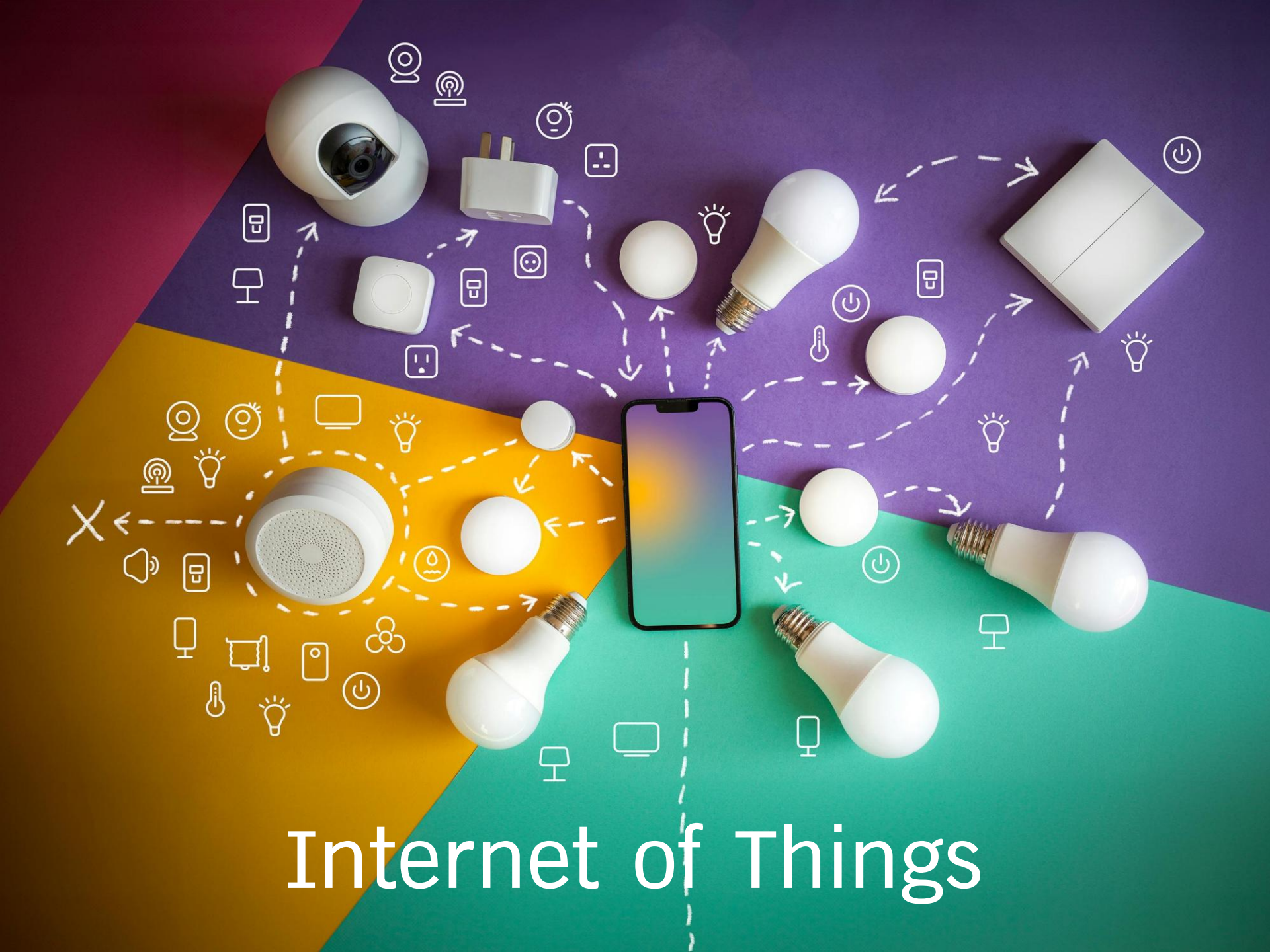

Internet of Things

What is Internet of Things

นิยามของ Internet of Things ที่เผยแพร่ในเอกสารของ IEEE เมื่อ 27 May 2015

An IoT is a network that connects uniquely identifiable **"Things"** to the Internet.

The **"Things"** have sensing/actuation and potential programmability capabilities.

Through the exploitation of unique identification and sensing information about the **"Thing"** can be collected and the state of the **"Things"** can be changed from anywhere, anytime, by anything.

IoT คือเครือข่ายที่เชื่อมต่อ "**สรรพสิ่ง**" ที่สามารถระบุตัวตนได้อย่างเป็นเอกลักษณ์ เข้ากับอินเทอร์เน็ต

"**สรรพสิ่ง**" เหล่านี้มีความสามารถในการรับรู้/ตอบสนอง และอาจมีความสามารถในการตั้งโปรแกรมได้

ด้วยการใช้ประโยชน์จากการระบุตัวตนที่เป็นเอกลักษณ์และข้อมูลการรับรู้ สามารถรวบรวมข้อมูลเกี่ยวกับ "**สรรพสิ่ง**" นั้นได้ และสามารถเปลี่ยนแปลงสถานะของ "**สรรพสิ่ง**" เหล่านั้นได้จากที่ไหนก็ได้ เมื่อไหร่ก็ได้ โดยอะไรก็ได้

An **IoT** is a network that connects uniquely identifiable "Things" to the Internet.
The "Things" have sensing/actuation and potential programmability capabilities.
Through the exploitation of unique identification and sensing information about the "Thing"
can be collected and the state of the "Things" can be changed from anywhere, anytime, by anything.

1 "Things"

Physical devices in the real world



2 Unique Identification

Each thing has a unique ID

- **QR Code** ID: IOT-0001
- **Serial Number** SN: 2026-00345
- **RFID** UID: A1B2C3D4
- **MAC Address** 00:1A:2B:3C:4D:5E
- **SIM / eSIM** IMSI: 520030123456789

3 Connection to the Internet

Networks that connect things



4 Sensing Information Collection

Collect data from the things

- **Temperature** 25.6 °C
- **Humidity** 58 %
- **Location** 13.736°N, 100.528°E
- **Status** ON / OFF
- **Condition** Normal / Abnormal
- ...

5 Actuation & Control

Change the state of things

- **Turn ON / OFF** (eg. lights)
- **Lock / Unlock** (eg. smart lock)
- **Start / Stop** (eg. motor)
- **Open / Close** (eg. valve)
- **Set Parameter** (eg. temperature)
- ...

6 Access from Anywhere, Anytime, by Anything

Users, systems or AI can monitor and control the things from **anywhere**, anytime, using any device.



- **AI / Application**
- **Other Systems** (API / Automation)

IoT Platform

Store, analyze, visualize and manage data



Sensing

Temperature | Light | Motion | Location | ...

Actuation

Motor | Valve | Light | Door Lock | ...

Programmability

Update logic | Set rules | Remote control | ...

Real-world Impact

Smart City | Smart Home | Smart Industry
Smart Agriculture | Smart Healthcare | ...

What is Internet of Things

Internet of Things features

- Interconnection of Things
- Connection of Things to the Internet
- Uniquely Identifiable Things
- Ubiquity
- Sensing/Actuation capability
- Embedded intelligence
- Interoperable Communication Capability
- Self configurability
- Programmability

Internet of Things features

1 Interconnection of Things

- Many physical things (devices, sensors, actuators) are interconnected with each other and with systems.



2 Connection of Things to the Internet

- Things connect to the Internet via Wi-Fi, Cellular (4G/5G), LPWAN (LoRa/NB-IoT), Ethernet, Satellite, etc.



3 Uniquely Identifiable Things

- Each thing has a unique identity (e.g., ID, UID, MAC address, QR Code, RFID, Digital Twin).



Device ID: 3F:5A:9C:1D:2E:7B
Serial No: SN-2026-00123
QR Code:
Digital Twin Twin

4 Ubiquity

- IoT can be deployed anywhere, anytime (homes, cities, factories, remote areas) and accessible globally.





5 Sensing/Actuation capability

- Things can sense the physical world (temperature, light, motion, location, etc.) and act on the environment (e.g., control lights, motors, valves).



6 Embedded intelligence

- Devices have built-in processing, local analytics, and decision-making (Edge AI / TinyML).



- Local data processing
- AI inference
- Event detection
- Real-time decision
- Reduce latency
- Save bandwidth

7 Interoperable Communication Capability

- Things can communicate using standard protocols and work with heterogeneous devices and platforms.

Wi-Fi	Bluetooth	Zigbee
LoRa / LoRaWAN	NB-IoT	5G 5G
Ethernet	MQTT	CoAP
HTTP/REST	OPC UA	Modbus

8 Self configurability

- Devices can automatically discover, configure and join the network.



- [x] Auto discovery
- [x] Auto configuration
- [x] Join network
- [x] Update settings
- [x] Self-healing (reconnect on failure)

9 Programmability

- Things can be programmed to change behavior, update functionality, and support new applications.



</>

- Remote programming
- Firmware update (OTA)
- New features
- Adaptive behavior
- Support multiple applications

เทคโนโลยีการเชื่อมต่อของสรรพสิ่ง

- **นิยามโดย กระทรวงดิจิทัล**
- Internet of Things หรือ IoT คือ สภาพแวดล้อมอันประกอบด้วยสรรพสิ่งที่สามารถสื่อสารและเชื่อมต่อกันได้ผ่านโพรโทคอลการสื่อสารทั้งแบบใช้สายและไร้สาย โดยสรรพสิ่งต่างๆ มีวิธีการระบุตัวตนได้ รับรู้บริบทของสภาพแวดล้อมได้ และมีปฏิสัมพันธโต้ตอบและทำงานรวมกันได้

- IoT จะเปลี่ยนรูปแบบและกระบวนการผลิตในภาคอุตสาหกรรมไปสู่ยุคใหม่ หรือที่เรียกว่า Industry 4.0 ที่จะอาศัยการเชื่อมต่อสื่อสารและทำงานร่วมกันระหว่างเครื่องจักร มนุษย์ และข้อมูล เพื่อเพิ่มอำนาจในการตัดสินใจที่รวดเร็วและมีความถูกต้องแม่นยำสูง

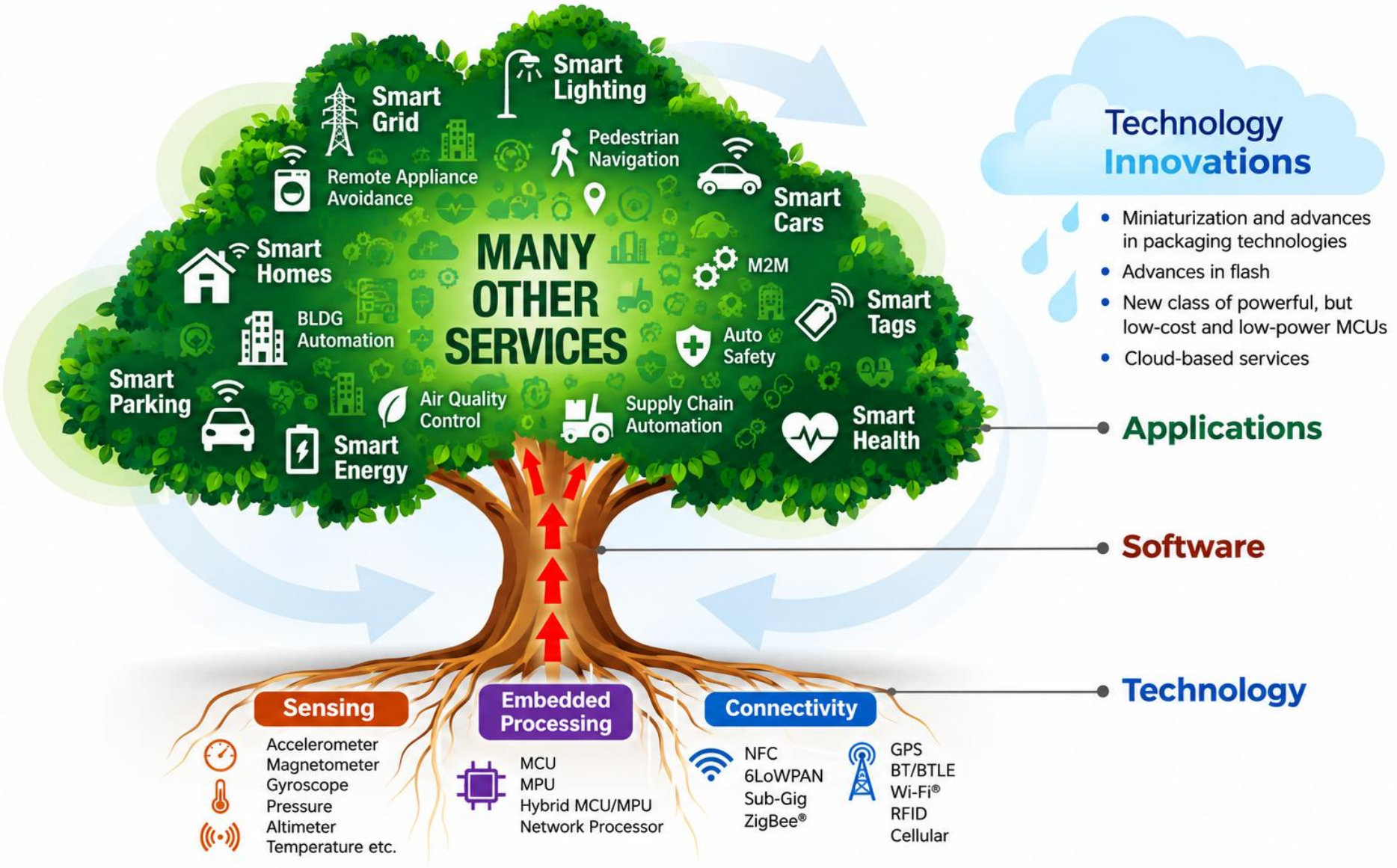
- โดยเทคโนโลยีที่ทำให้ IoT เกิดขึ้นได้จริงและสร้างผลกระทบในวงกว้างได้ แบ่งออกเป็นสามกลุ่ม ได้แก่

  1) **เทคโนโลยีที่ช่วยให้สรรพสิ่งรับรู้ข้อมูล** ในบริบทที่เกี่ยวข้อง เช่น เซ็นเซอร์

  2) **เทคโนโลยีที่ช่วยให้สรรพสิ่งมีความสามารถในการสื่อสาร** เช่น ระบบสมองกลฝังตัว รวมถึงการสื่อสารแบบไร้สายที่ใช้พลังงานต่ำ อาทิ Zigbee, 6LowPAN, Low-power Bluetooth

  3) **เทคโนโลยีที่ช่วยให้สรรพสิ่งประมวลผลข้อมูล** ในบริบทของตน เช่น เทคโนโลยีการประมวลผลแบบคลาวด์ และเทคโนโลยีการวิเคราะห์ข้อมูลขนาดใหญ่ หรือ Big Data Analytics

ที่มา : แผนพัฒนาดิจิทัล กระทรวงดิจิทัล



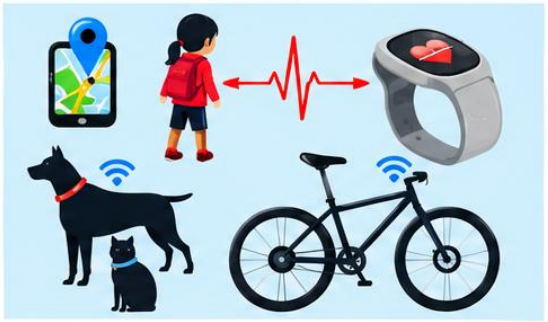
The IoT: Different Services, Technologies, Meanings for Everyone [1]

IoT Use case

Smart City Applications



Personal IoT Applications



Smart Grid & Smart Metering



Industrial Assets Monitoring



Critical Infrastructure Monitoring



Agriculture



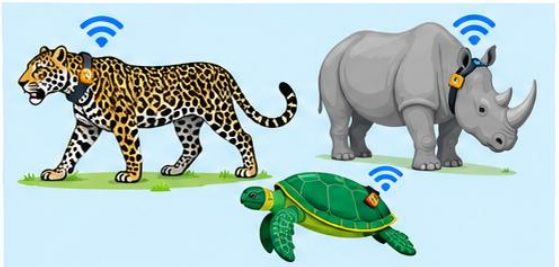
Home Automation & Safety



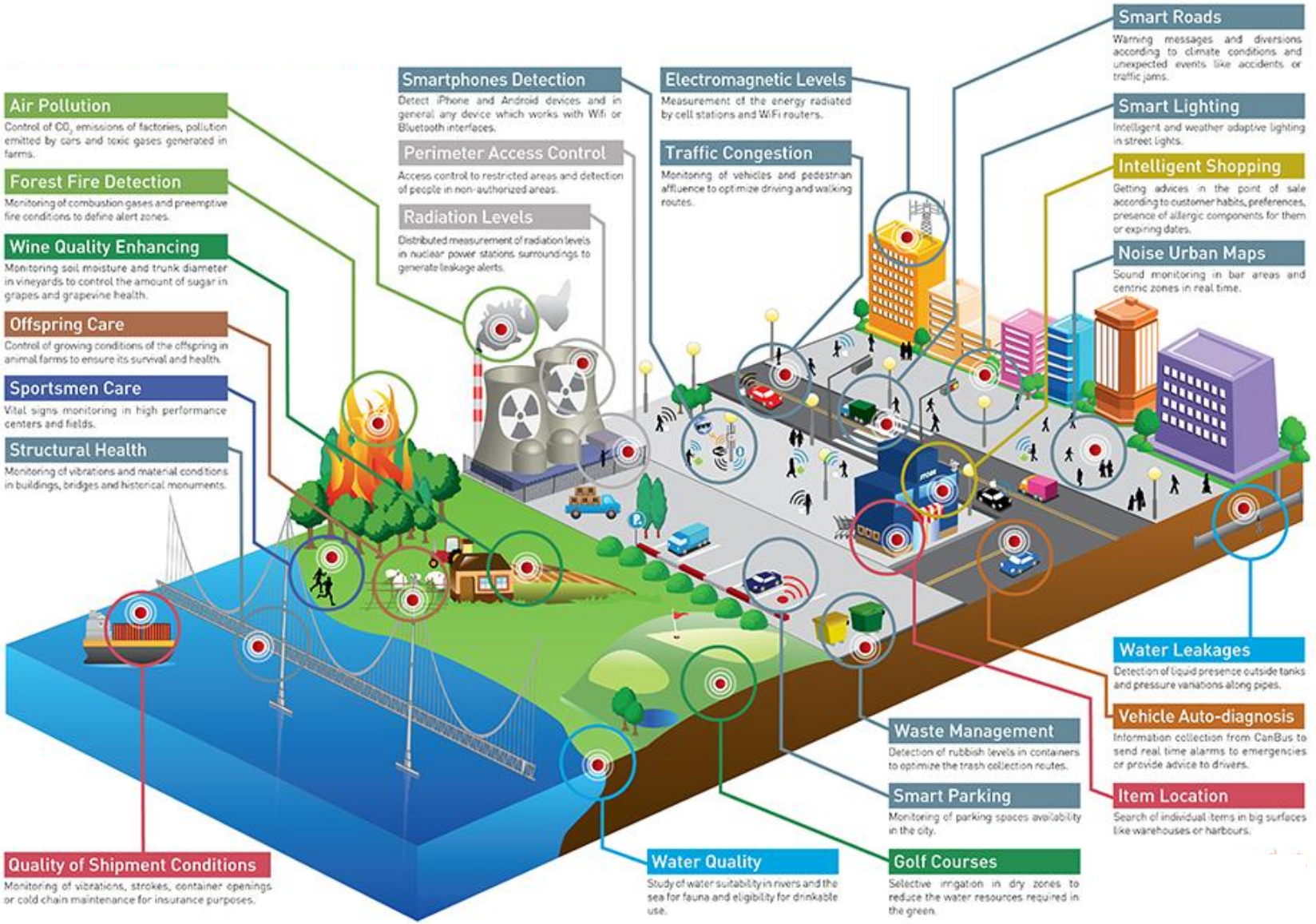
Logistics



Wildlife Monitoring & Tracking



IoT Use case : Smart-X Application



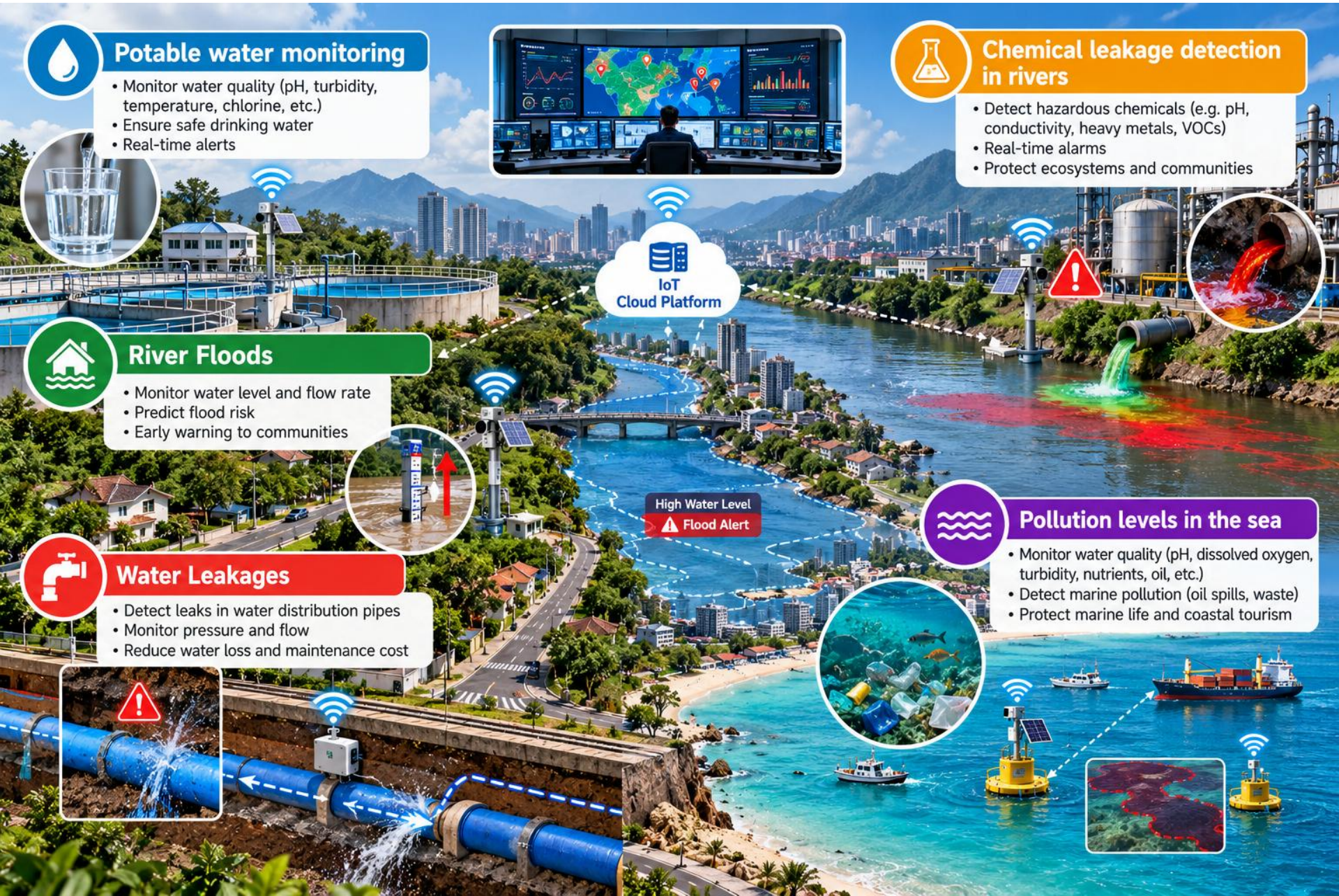
IoT Use case : Smart City



IoT Use case : Smart Environment



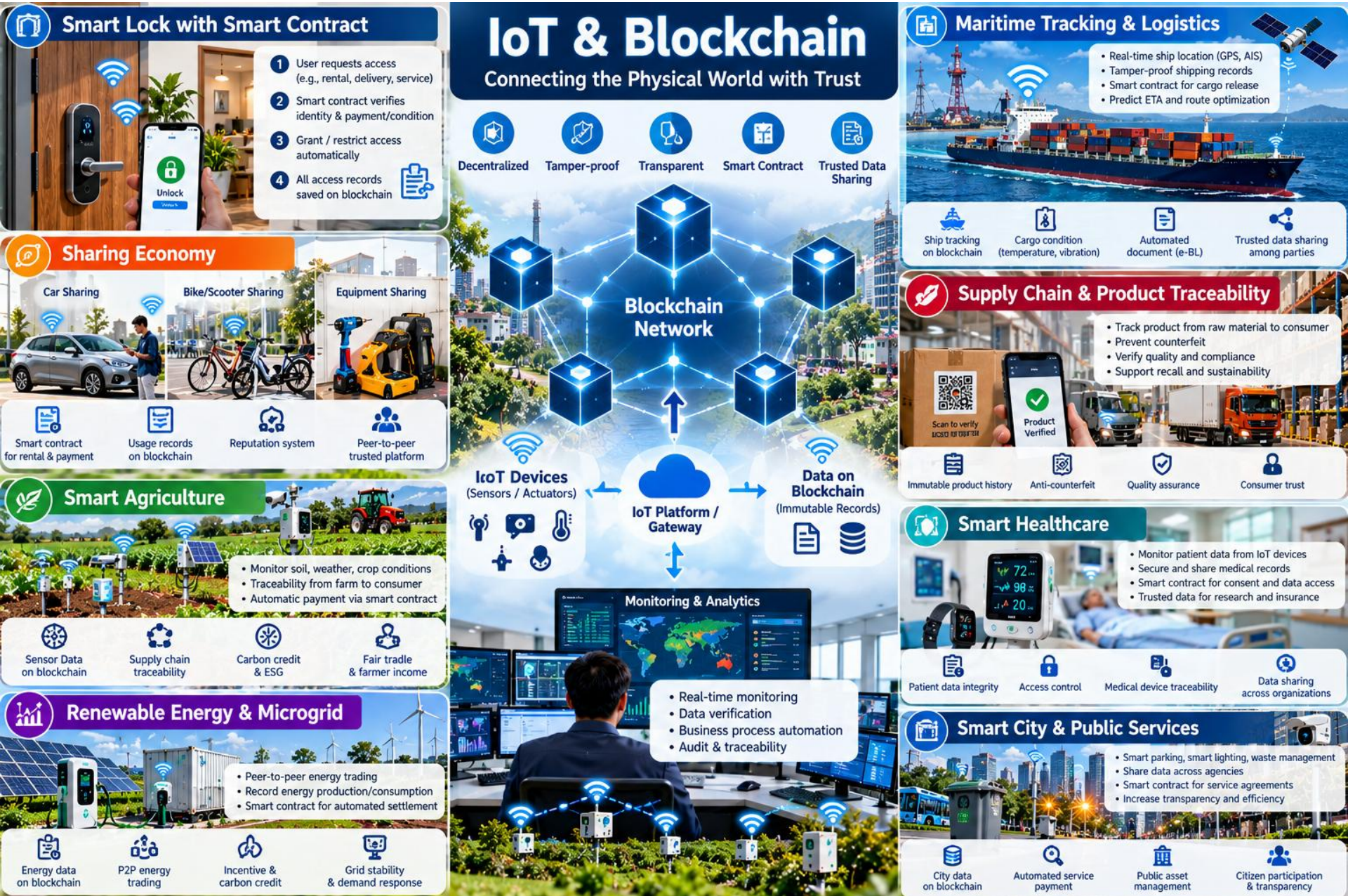
IoT Use case : Smart water



IoT Use case : Smart Health



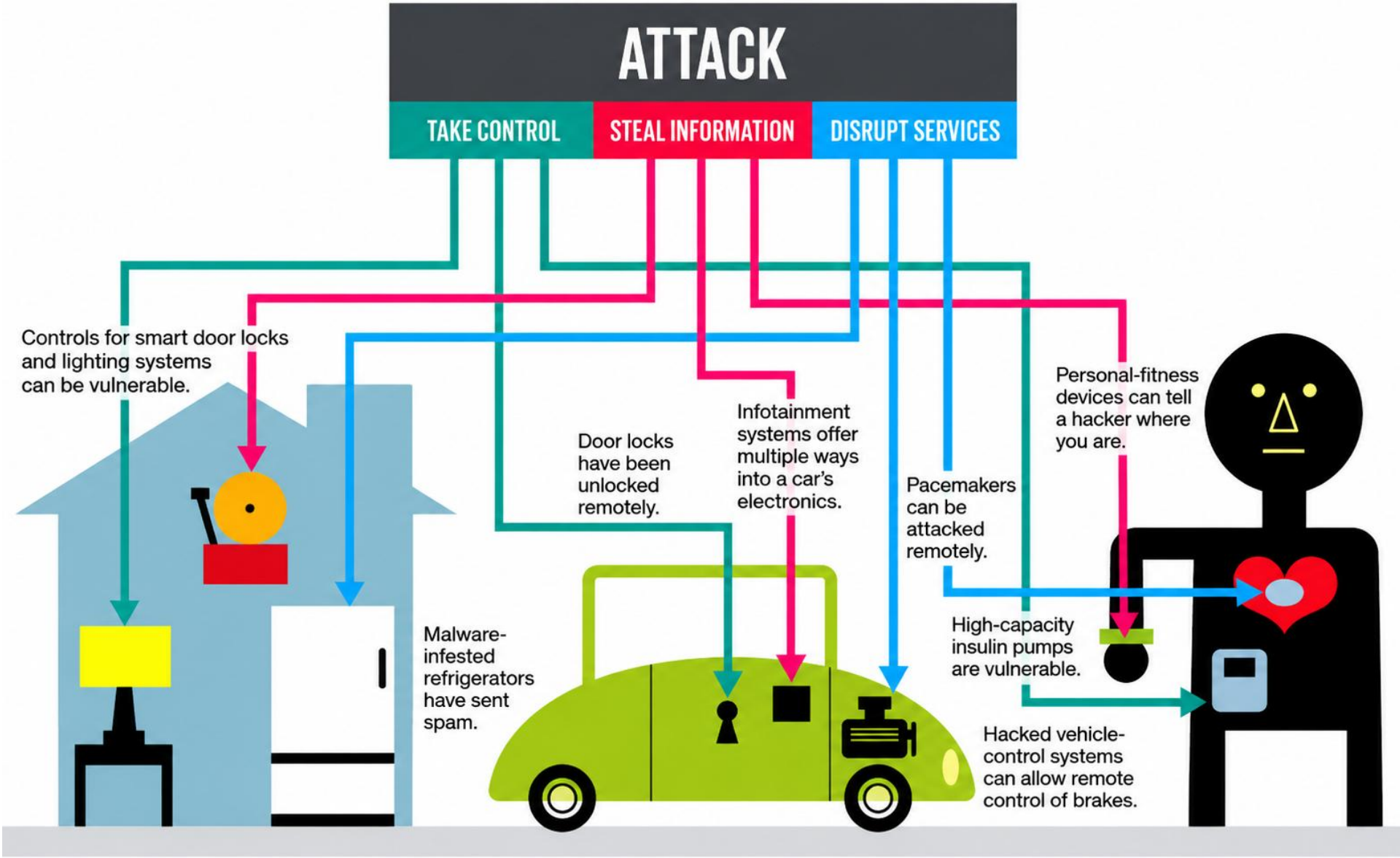
ตัวอย่าง IoT และ Blockchain

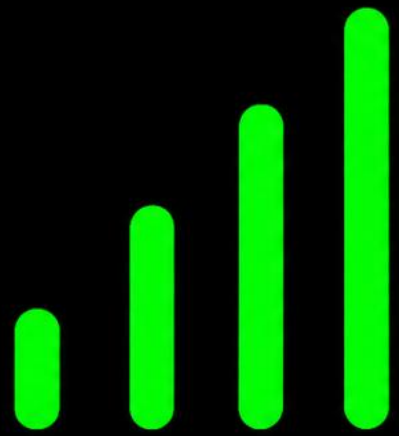


ตัวอย่าง IoT และ AI



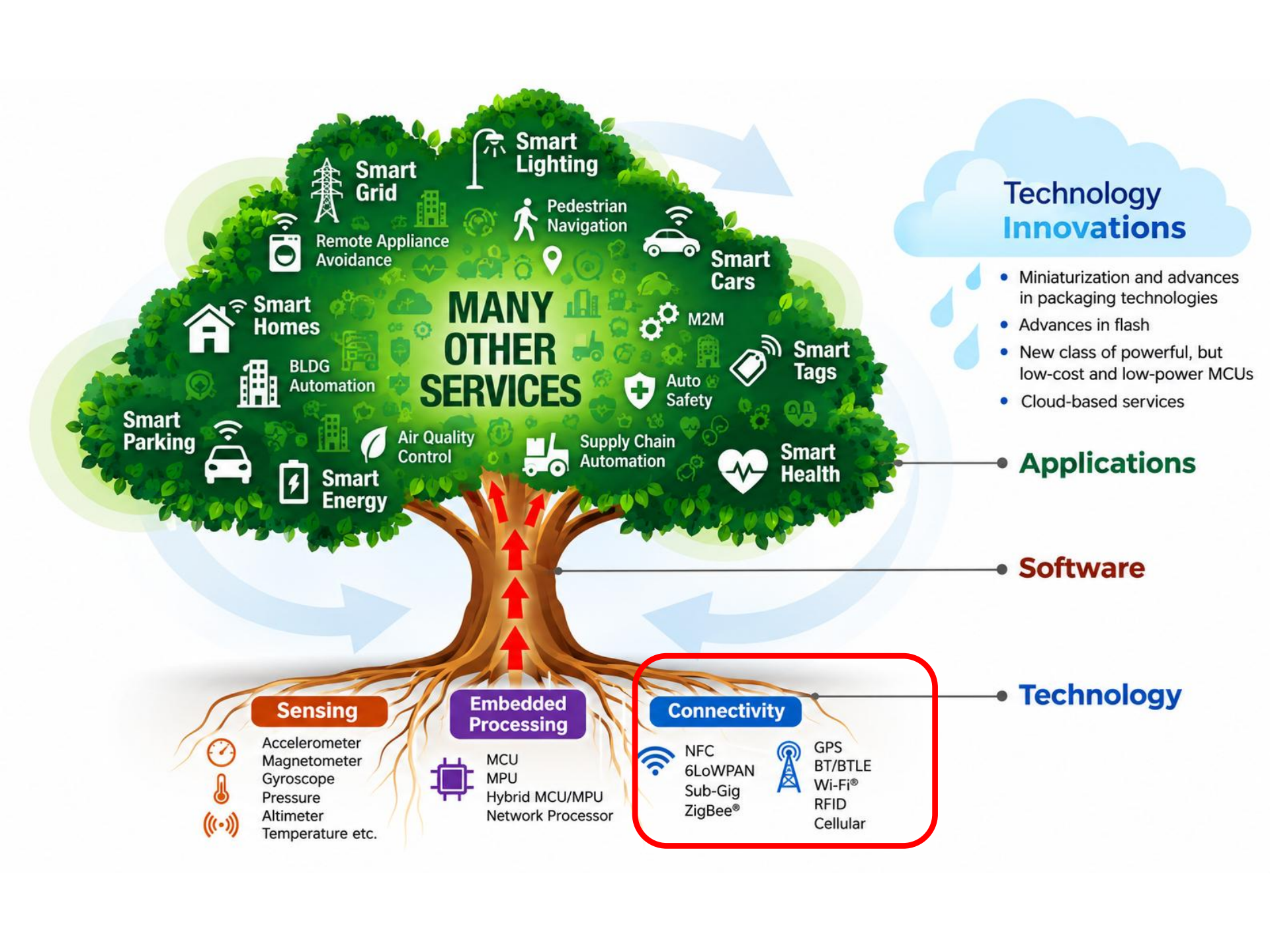
Security ของ IoT





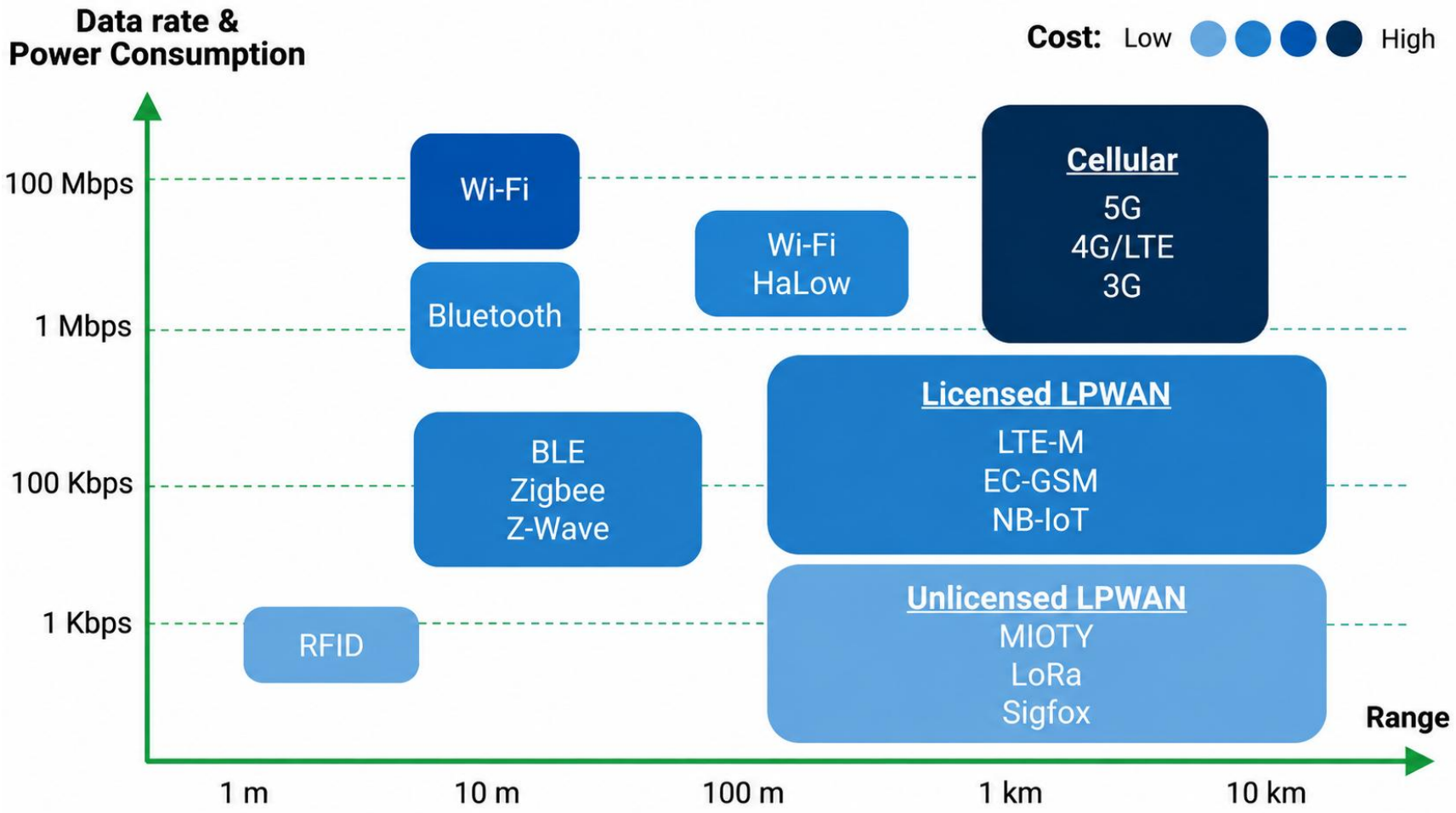


Connectivity


Smart Grid
Smart Lighting
Pedestrian Navigation
Remote Appliance Avoidance
Smart Cars
Smart Homes
MANY OTHER SERVICES
M2M
Smart Tags
BLDG Automation
Auto Safety
Smart Parking
Air Quality Control
Supply Chain Automation
Smart Health
Smart Energy
Technology Innovations
Miniaturization and advances in packaging technologies
Advances in flash
New class of powerful, but low-cost and low-power MCUs
Cloud-based services
Applications
Software
Technology
Sensing
Accelerometer
Magnetometer
Gyroscope
Pressure
Altimeter
Temperature etc.
Embedded Processing
MCU
MPU
Hybrid MCU/MPU
Network Processor
Connectivity
NFC
6LoWPAN
Sub-Gig
ZigBee®
GPS
BT/BTLE
Wi-Fi®
RFID
Cellular


Short Range
Long Range Unlicensed
sigfox
LoRa
Long Range Licensed
NB IoT
LTE
connectivity options for things

Wireless Technology for IoT



Internet of Things Protocol



Protocol

- Protocol แปลว่า กฎเกณฑ์ / ข้อตกลง / วิธีปฏิบัติที่กำหนดไว้
  - Company Protocol = กฎเกณฑ์หรือแนวปฏิบัติของบริษัท
  - Communication Protocol = กฎเกณฑ์ในการสื่อสารระหว่างอุปกรณ์
  - Network Protocol = กฎเกณฑ์ในการรับส่งข้อมูลบนเครือข่าย
  - HTTP Protocol = กฎเกณฑ์ในการสื่อสารระหว่าง Web Client และ Web Server โดยสามารถรับ-ส่งข้อมูลได้หลายรูปแบบ เช่น HTML, รูปภาพ, Video, CSS, JavaScript, JSON
  - IoT Protocol คือ กฎเกณฑ์หรือมาตรฐานที่กำหนดว่าอุปกรณ์ IoT จะสื่อสารและรับ–ส่งข้อมูลกันอย่างไร
    - ตัวอย่างเช่น MQTT , CoAP , AMQP

MQTT Protocol

- MQTT : Message Queuing Telemetry Transport
- Invented by Andy Stanford Clark (IBM) and Arlen Nipper (Eurotech) in 1999
- Originally envisioned for use over Satellite link from an oil pipeline

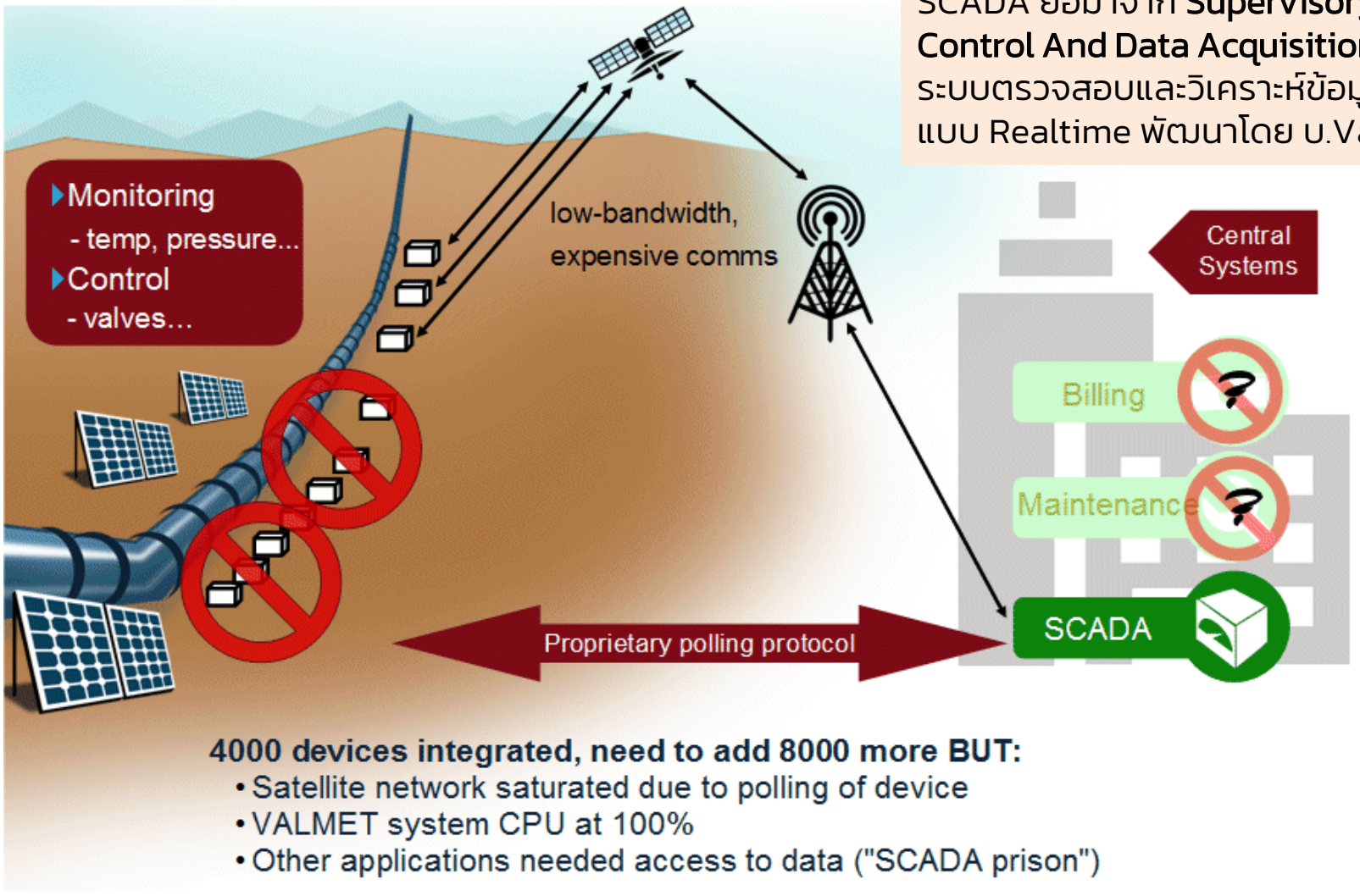


Use case:

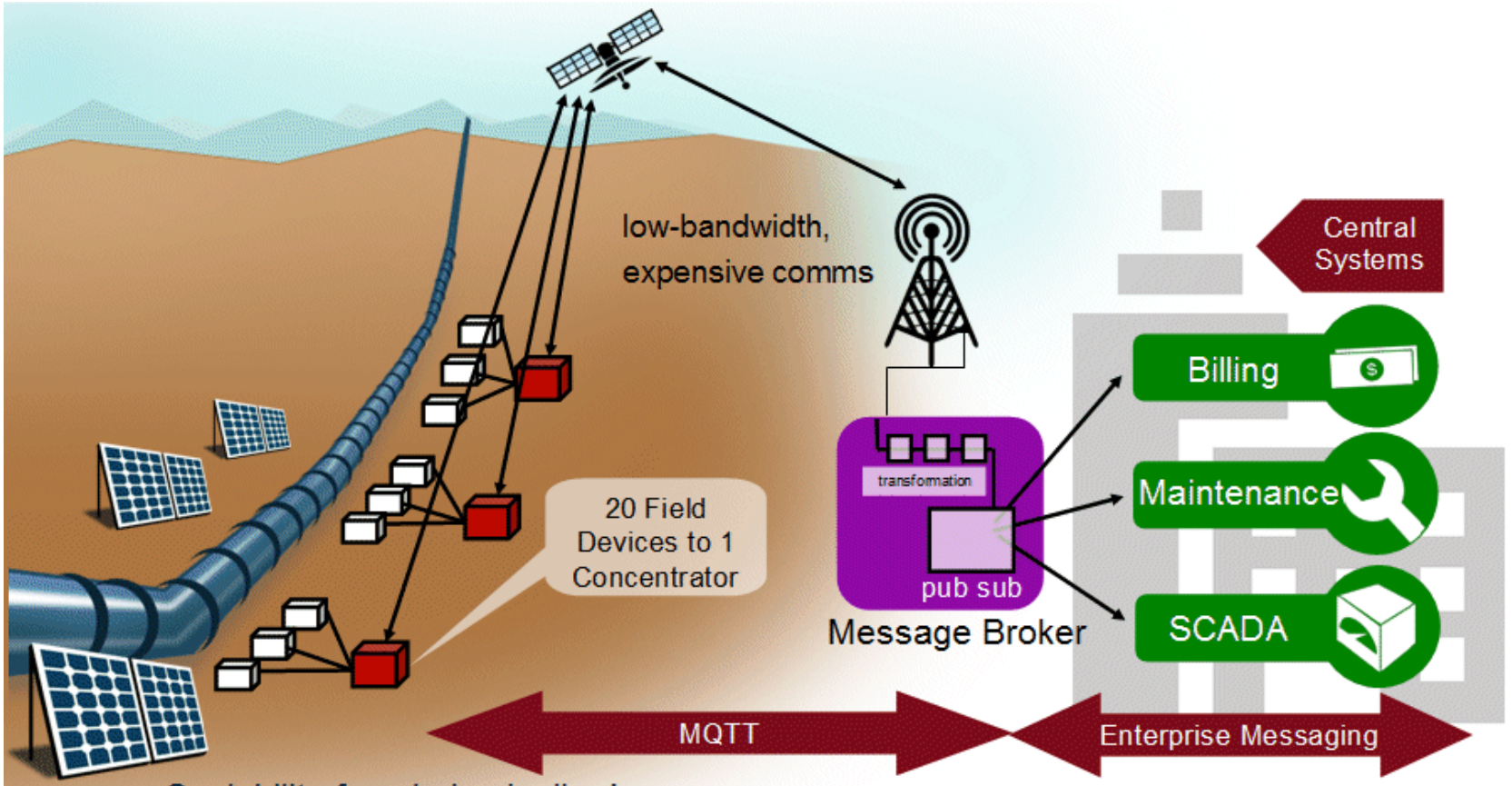
- 30.000 devices
- 17.000 km pipeline
- Remote monitoring
- Remote control
- Uses satellite links
- Bandwidth is **very** expensive

ที่มา : MQTT Protocol

SCADA ย่อมาจาก **Supervisory Control And Data Acquisition** เป็นระบบตรวจสอบและวิเคราะห์ข้อมูลแบบ Realtime พัฒนาโดย บ.Valmet



ที่มา : MQTT Protocol

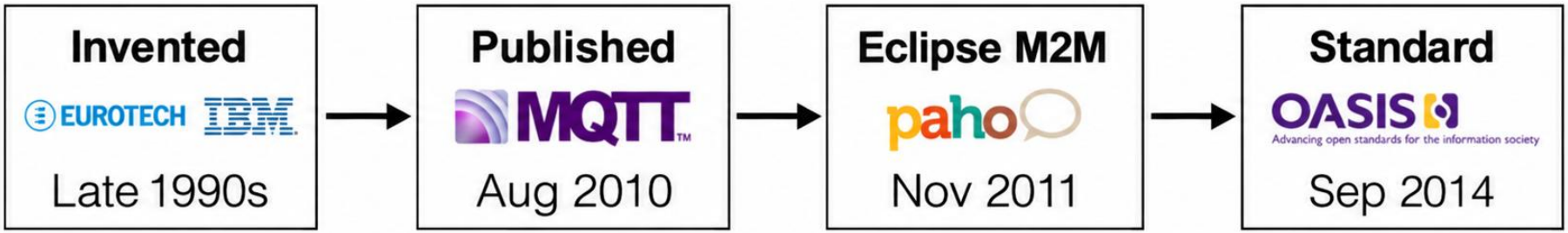


Scalability for whole pipeline!
Network traffic much lower - events pushed to/from devices and report by exception
Network cost reduced
Lower CPU utilization
Broken out of the SCADA prison – data accessible to other applications

ลักษณะของ MQTT Protocol

MQTT a lightweight protocol for IoT messaging

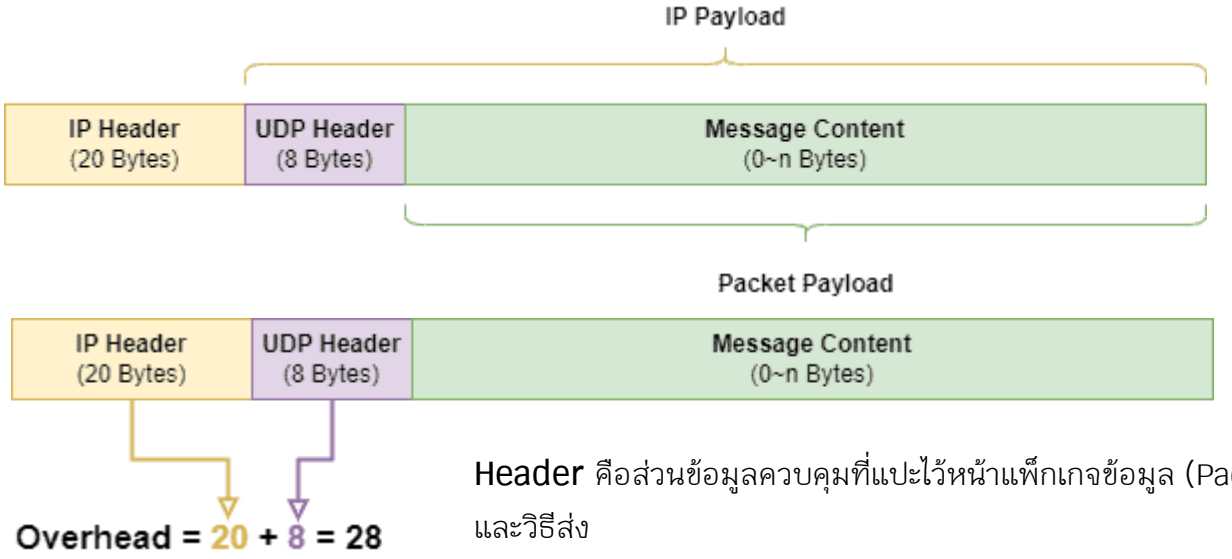
- **open** — open spec, standard — 40+ client implementations
- **lightweight** — minimal overhead — efficient format — tiny clients (kb)
- **reliable** — QoS for reliability on unreliable networks
- **simple** — 43-page spec — connect + publish + subscribe



ลักษณะของ MQTT Protocol

- **MQTT** มี Header ขนาดเล็ก (ขั้นต่ำ 2 bytes) จึงมี Protocol Overhead ต่ำ ทำให้เหมาะกับอุปกรณ์ IoT ที่มี Bandwidth และทรัพยากรจำกัด และใช้ **Persistent TCP Connection** (เปิด TCP connection ค้างไว้ กับ MQTT Broker เพื่อส่ง/รับข้อความต่อเนื่อง)

- Note!! ข้อแตกต่าง Header & Overhaed

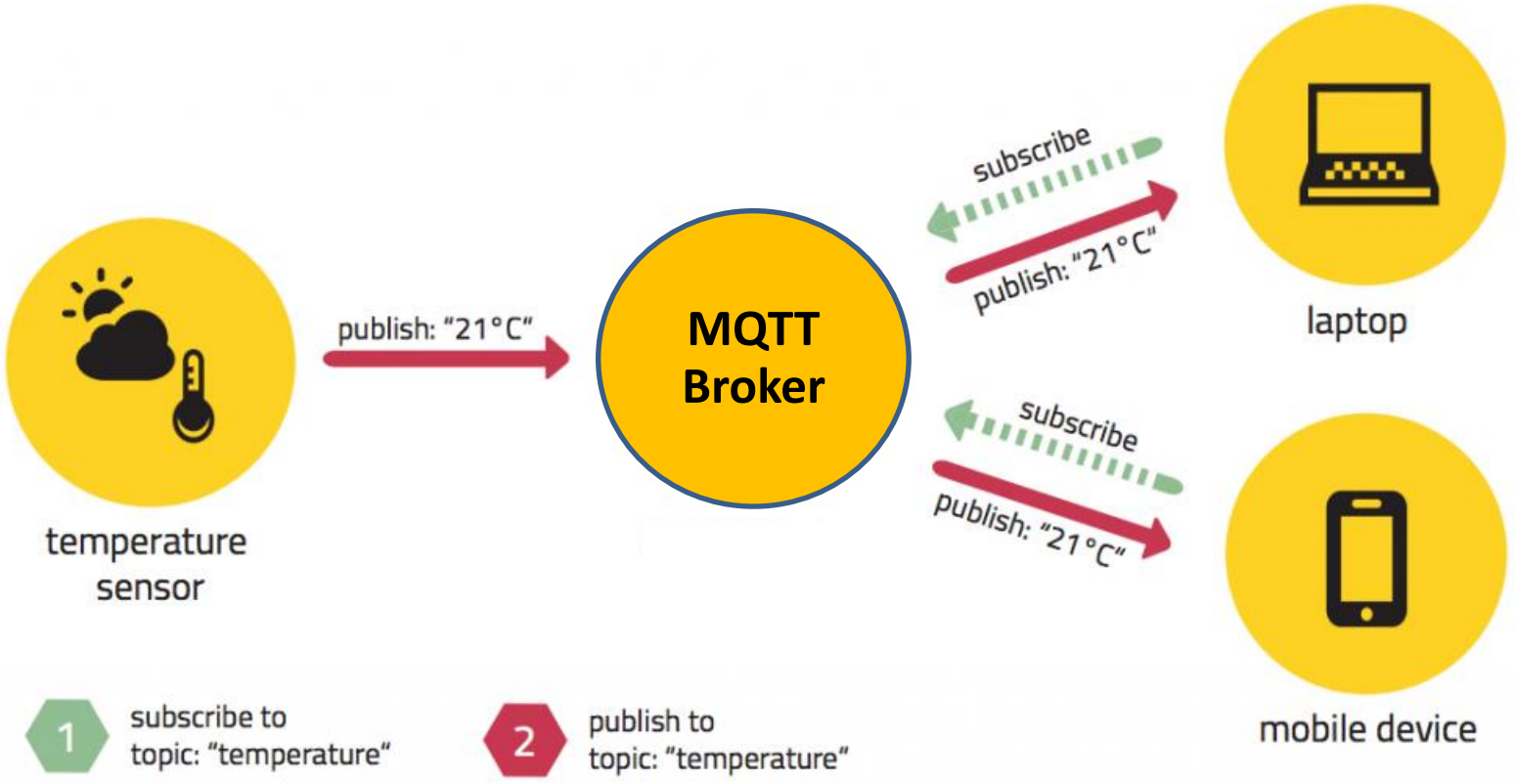


Header คือส่วนข้อมูลควบคุมที่แปะไว้หน้าแพ็กเกจข้อมูล (Packet) เพื่อบอกเส้นทางและวิธีส่ง

Overhead คือปริมาณข้อมูลส่วนเกินทั้งหมด (ซึ่งรวม Header และ Trailer) ที่ไม่ใช่ข้อมูลจริงของผู้ใช้ (Payload)

The publish/subscribe pattern

Bi-directional , Asynchronous "**Push**" Communication



MQTT Protocol

MQTT

simple to implement

Connect

Subscribe

Publish

Unsubscribe

Disconnect

```
client = new Messaging.Client(hostname, port, clientId)
client.onMessageArrived = messageArrived;
client.onConnectionLost = connectionLost;
client.connect({ onSuccess: connectionSuccess });

function connectionSuccess() {
     client.subscribe("planets/earth");
     var msg = new Messaging.Message("Hello world!");
     msg.destinationName = "planets/earth";
     client.publish(msg);
}

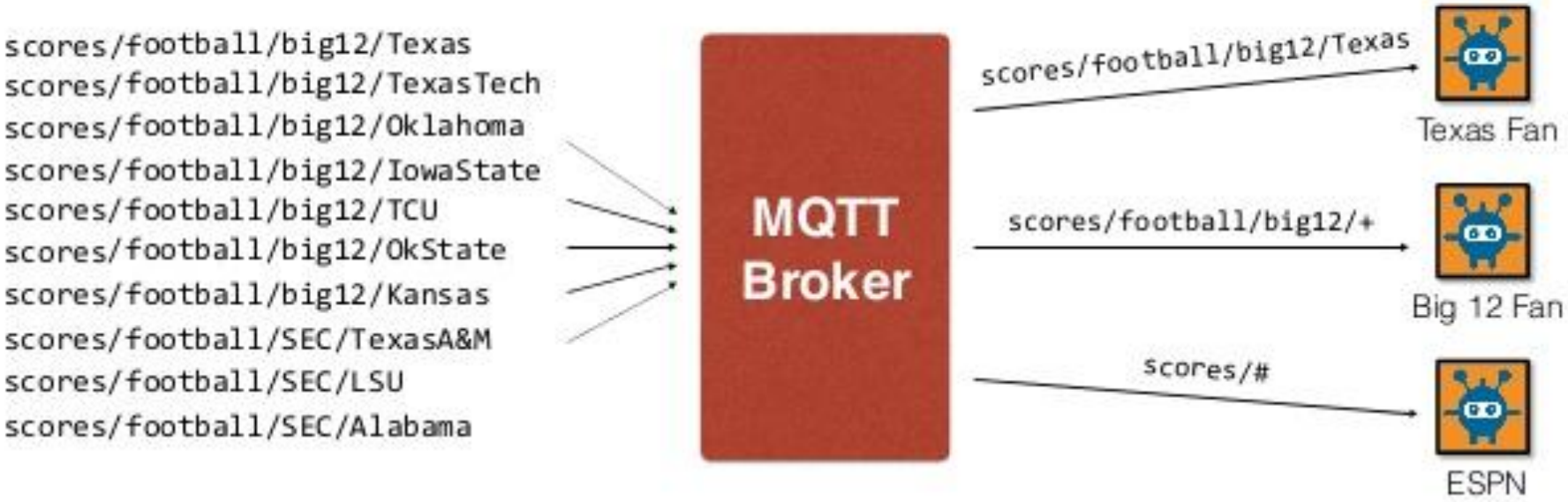
function messageArrived(msg) {
     console.log(msg.payloadString);
     client.unsubscribe("planets/earth");
     client.disconnect();
}
```

Eclipse Paho JavaScript MQTT client

Subscribe

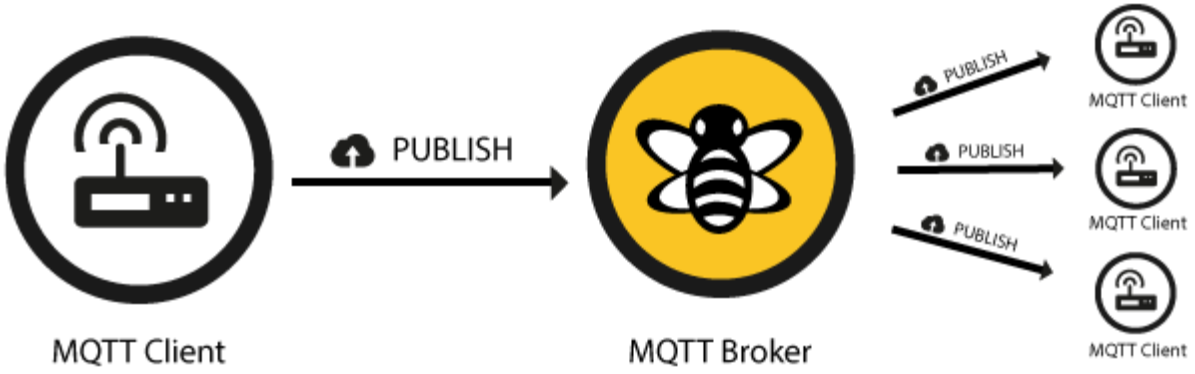
MQTT

allows wildcard subscriptions



single level wildcard: **+** multi-level wildcard: **#**

Publish



MQTT-Packet:

PUBLISH

contains:	Example
`packetId` (always 0 for qos 0)	`4314`
`topicName`	`"topic/1"`
`qos`	`1`
`retainFlag`	`false`
`payload`	`"temperature:32.5"`
`dupFlag`	`false`

Publish

Topics
A topic is a UTF-8 string, which is used by the broker to filter messages for each connected client. A topic consists of one or more topic levels. Each topic level is separated by a forward slash (topic level separator).



QoS
A Quality of Service Level (QoS) for this message. The level (0,1 or 2) determines the guarantee of a message reaching the other end

Retain-Flag
This flag determines if the message will be saved by the broker for the specified topic as last known good value. New clients that subscribe to that topic will receive the last retained message on that topic instantly after subscribing. (ให้ Broker เก็บ Payload ล่าสุด ที่ส่งขึ้นไปไว้ด้วย ถ้ามีคนใหม่เข้ามา Subscribe ทีหลัง ก็จะได้รับข้อมูลนี้ด้วย)

Payload
This is the actual content of the message.

DUP flag
The duplicate flag indicates, that this message is a duplicate and is resent because the other end didn't acknowledge the original message. This is only relevant for QoS greater than 0

Packet Identifier
The packet identifier is a unique identifier between client and broker to identify a message in a message flow. This is only relevant for QoS greater than zero.

QoS (Quality of Service)

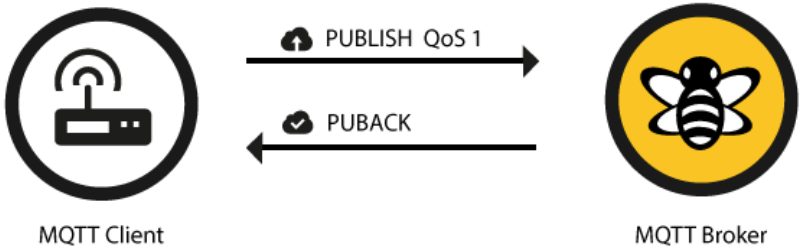
- **QoS 0 – at most once**

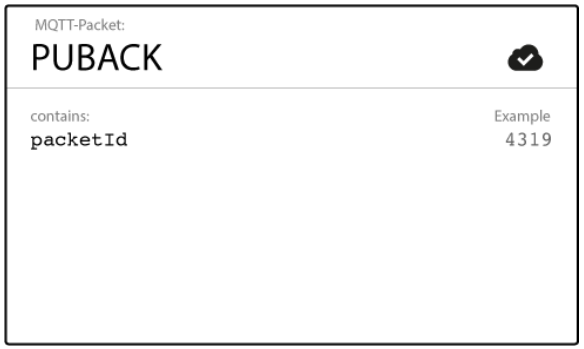
  The minimal level is zero and it guarantees a best effort delivery. A message won't be acknowledged by the receiver or stored and redelivered by the sender.



- **QoS 1 – at least once**

  it is guaranteed that a message will be delivered at least once to the receiver. The sender will store the message until it gets an acknowledgement in form of a PUBACK command message from the receiver.

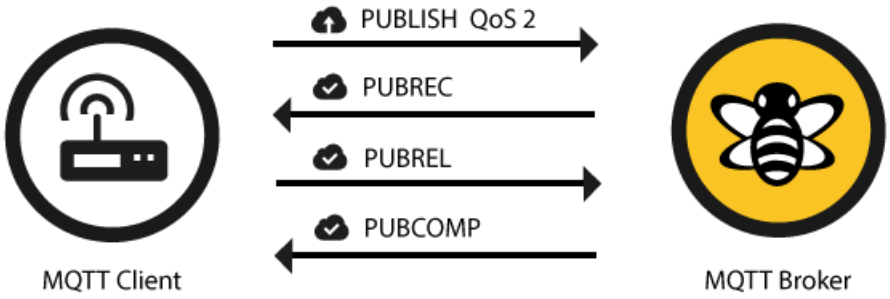




QoS

- **QoS 2 – Exactly once**

  The highest QoS is 2, it guarantees that each message is received only once by the counterpart. It is the safest and also the slowest quality of service level. The guarantee is provided by two flows there and back between sender and receiver.



MQTT-Packet:
PUBREC

contains: `packetId` Example 4320

Publish received

MQTT-Packet:
PUBREL

contains: `packetId` Example 4320

Publish release

MQTT-Packet:
PUBCOMP

contains: `packetId` Example 4320

Publish complete

QoS 0

- **Qos 0 :** ส่งไม่เกิน 1 ครั้ง (At most once) ไม่รับประกันว่าข้อความจะถึงปลายทาง
- **หลักการ**
  - ส่งครั้งเดียว
  - ไม่มีการยืนยันการรับ
- **ลักษณะ**
  - ข้อความอาจสูญหาย
  - ไม่มีการส่งซ้ำ
- **ขั้นตอน**



- ข้อดี
  - latency (เวลาในการสื่อสาร) ต่ำ เนื่องจาก Overhaed ต่ำ
  - ประหยัด bandwidth น้อย

QoS 1

- **QoS 1** : รับประกันว่าข้อความจะถูกส่ง **อย่างน้อยหนึ่งครั้ง** แต่มีโอกาสเกิด duplicate message (ข้อความซ้ำ)
- **หลักการ**
  - ต้องได้รับอย่างน้อย 1 ครั้ง
  - ถ้าไม่ได้รับ ACK จะส่งใหม่
- **ขั้นตอน**

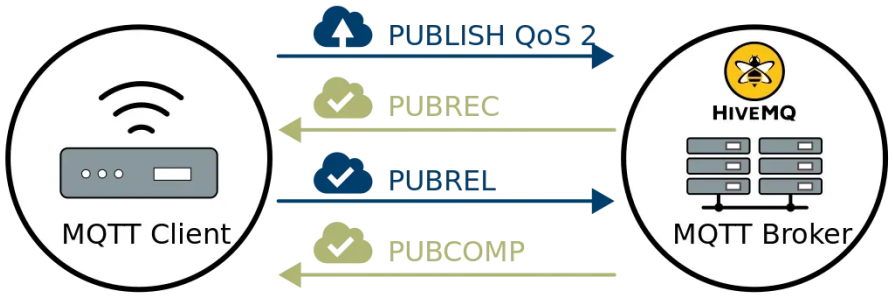


PUBACK = Publish Acknowledgement
“ยืนยันการรับข้อความ”

- **ผลที่อาจเกิด**
  - ข้อความ (message) อาจ **ซ้ำ**
- **ข้อดี**
  - reliability สูงกว่า QoS 0

QoS 2

- **QoS 2 :** รับประกันว่าข้อความจะถูกส่ง “เพียงครั้งเดียว” (Exactly once) และ ไม่เกิด Duplicate Message (ข้อความซ้ำ)
- **หลักการ**
  - รับประกัน **ส่งเพียงครั้งเดียว**
- **ขั้นตอน (มี 4 ขั้นตอน)**



PUBREC = Publish Received
“ฉันได้รับข้อความแล้ว”
PUBREL = Publish Release
“ตกลง ให้ดำเนินการกับข้อความนี้ได้”
PUBCOMP = Publish Complete
“ดำเนินการเสร็จสมบูรณ์แล้ว”

- **ข้อดี**
  - ไม่มี duplicate
- **ข้อเสีย**
  - latency (เวลาในการสื่อสาร) สูงกว่า **QoS 0 และ QoS 1**
  - network overhead สูง

สรุป

- 
  QoS 0: ส่งครั้งเดียว ข้อความอาจ**หายได้**

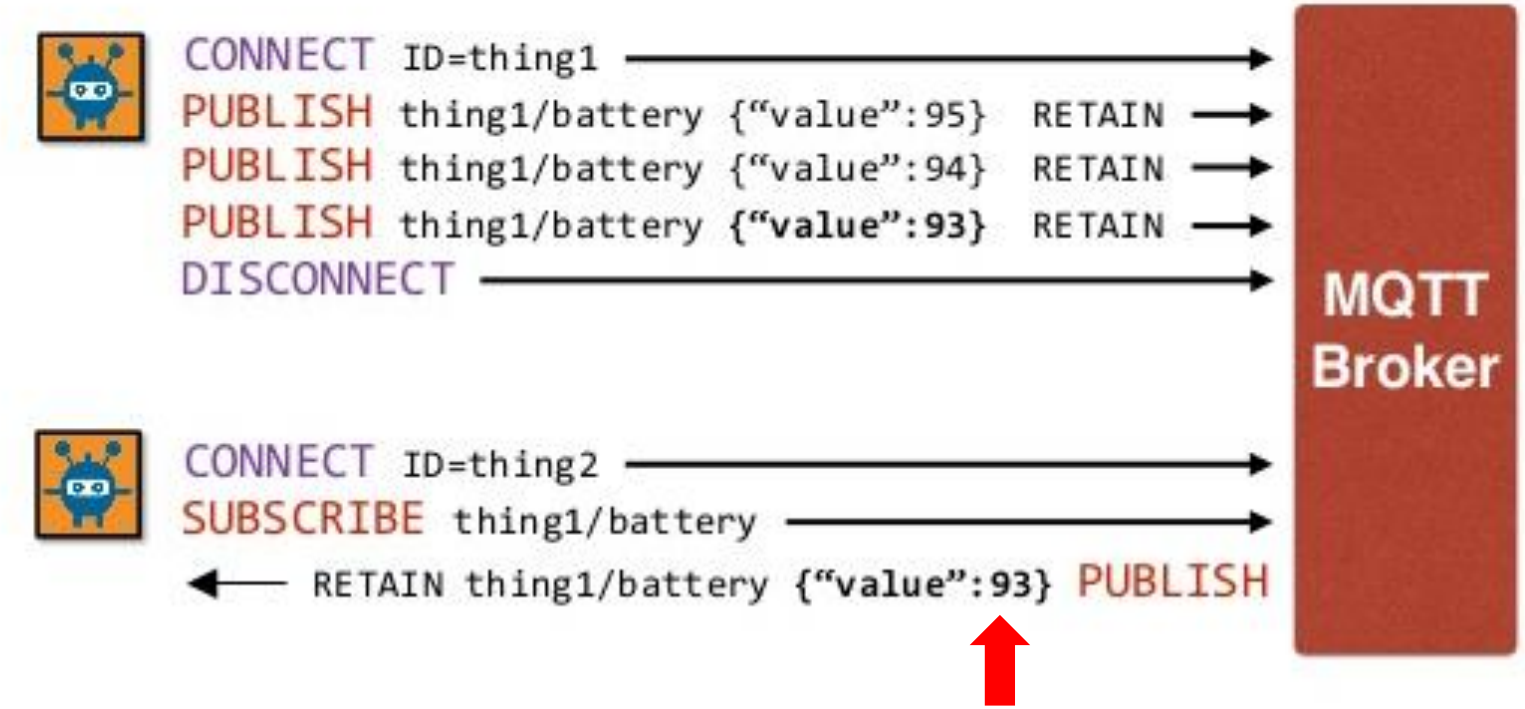
  
  QoS 1: ส่งจนกว่าจะได้รับ ACK ข้อความอาจ**ซ้ำได้**

  
  QoS 2: มีกระบวนการยืนยันหลายขั้น ข้อความ**ไม่หายและไม่ซ้ำ**

- Effective QoS = **min**(Publish QoS, Subscribe QoS)

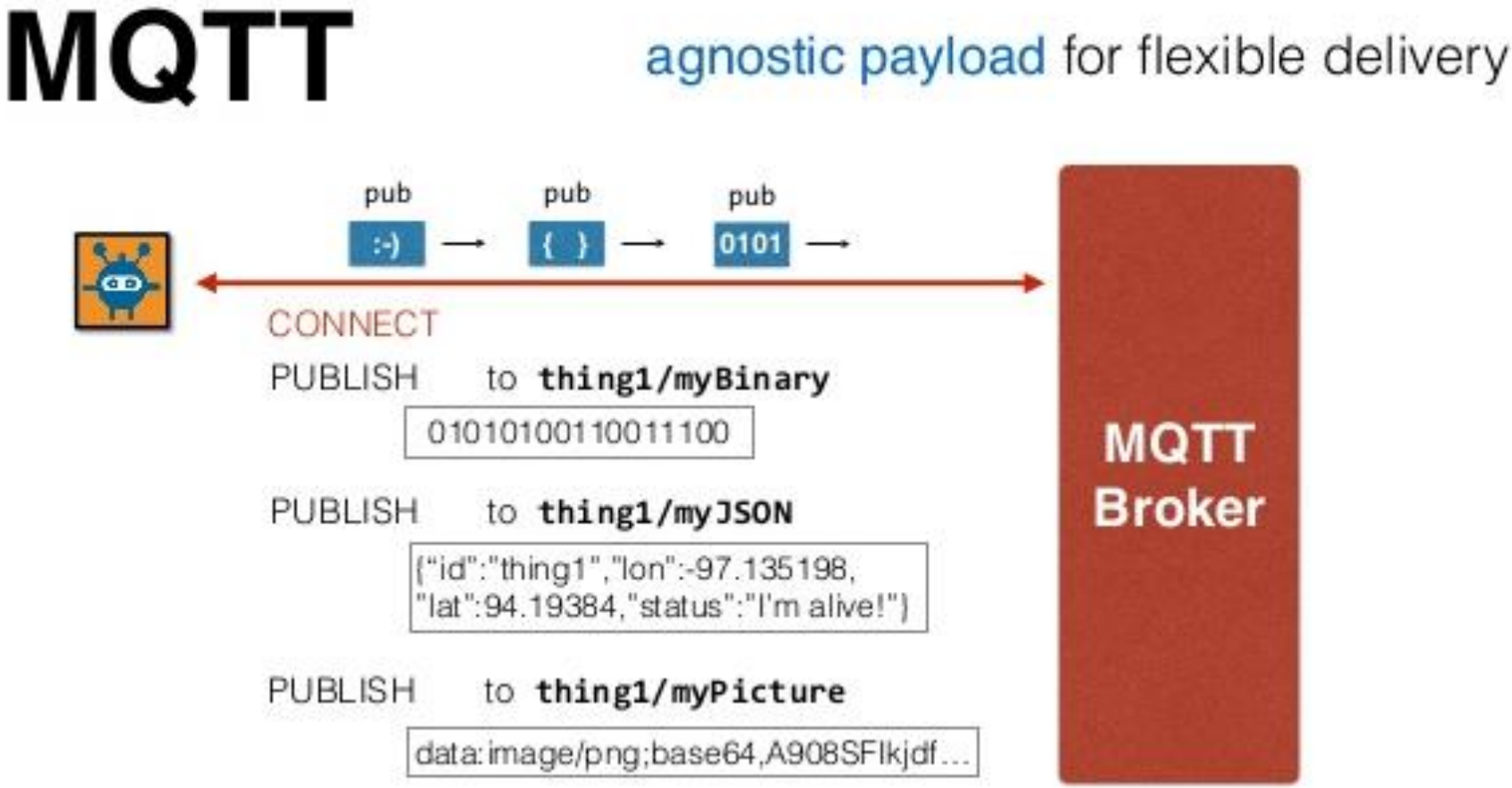
Retain messages

MQTT retained messages for last value caching



- Broker เก็บ message ล่าสุดของ topic
- Subscriber ใหม่จะได้รับข้อความนั้นทันที

Payload ส่งเป็น array

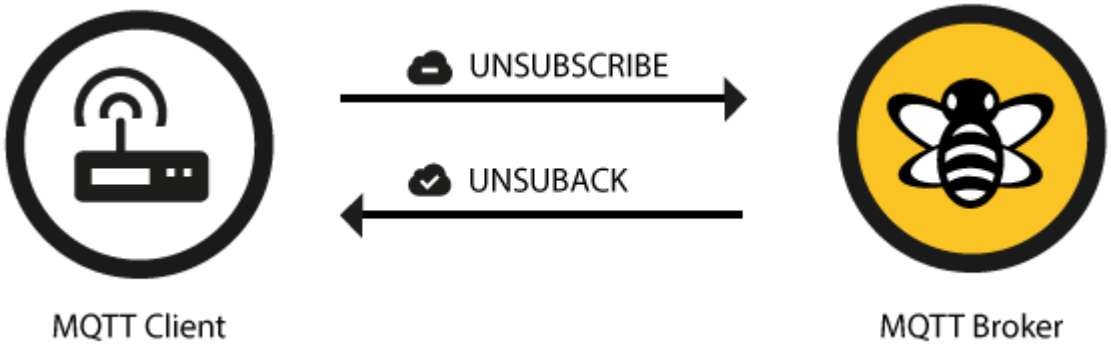


Agnostic แปลว่า ไม่ยึดติดหรือไม่ขึ้นกับสิ่งใดสิ่งหนึ่ง

Payload

- ในโปรโตคอล MQTT ข้อมูลที่ส่งใน **Payload** ไม่มีการกำหนดชนิดข้อมูลตายตัว (data type agnostic) โดย payload ถูกส่งในรูปแบบ **array of bytes (ชุดข้อมูลของ byte)** ดังนั้นจึงสามารถส่งข้อมูลได้หลายรูปแบบ เช่น
    - String
    - JSON
    - XML
    - ตัวเลข
    - รูปภาพ
    - ไฟล์

Unsubscribe



MQTT-Packet:

UNSUBSCRIBE

contains:

```
packetId
topic1
topic2
...
```

(list of topics)

Example

```
4315
"topic/1"
"topic/2"
...
```

MQTT-Packet:

UNSUBACK

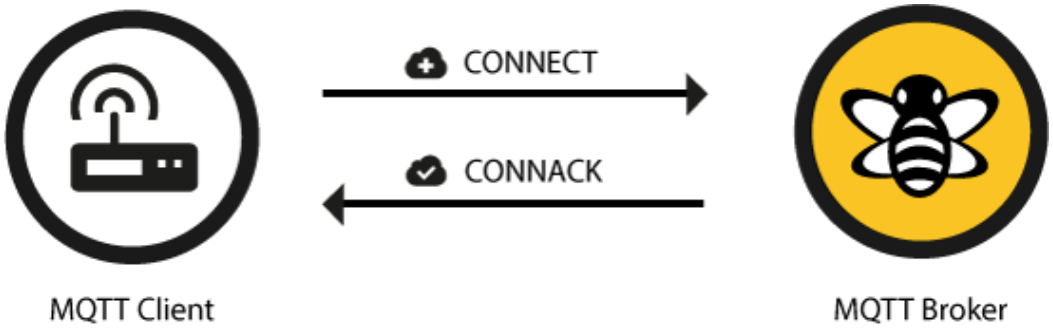
contains:

```
packetId
```

Example

```
4316
```

Connect & Disconnect



MQTT-Packet:
CONNECT

contains:	Example
clientId	"client-1"
cleanSession	true
username (optional)	"hans"
password (optional)	"letmein"
lastWillTopic (optional)	"/hans/will"
lastWillQos (optional)	2
lastWillMessage (optional)	"unexpected exit"
keepAlive	60

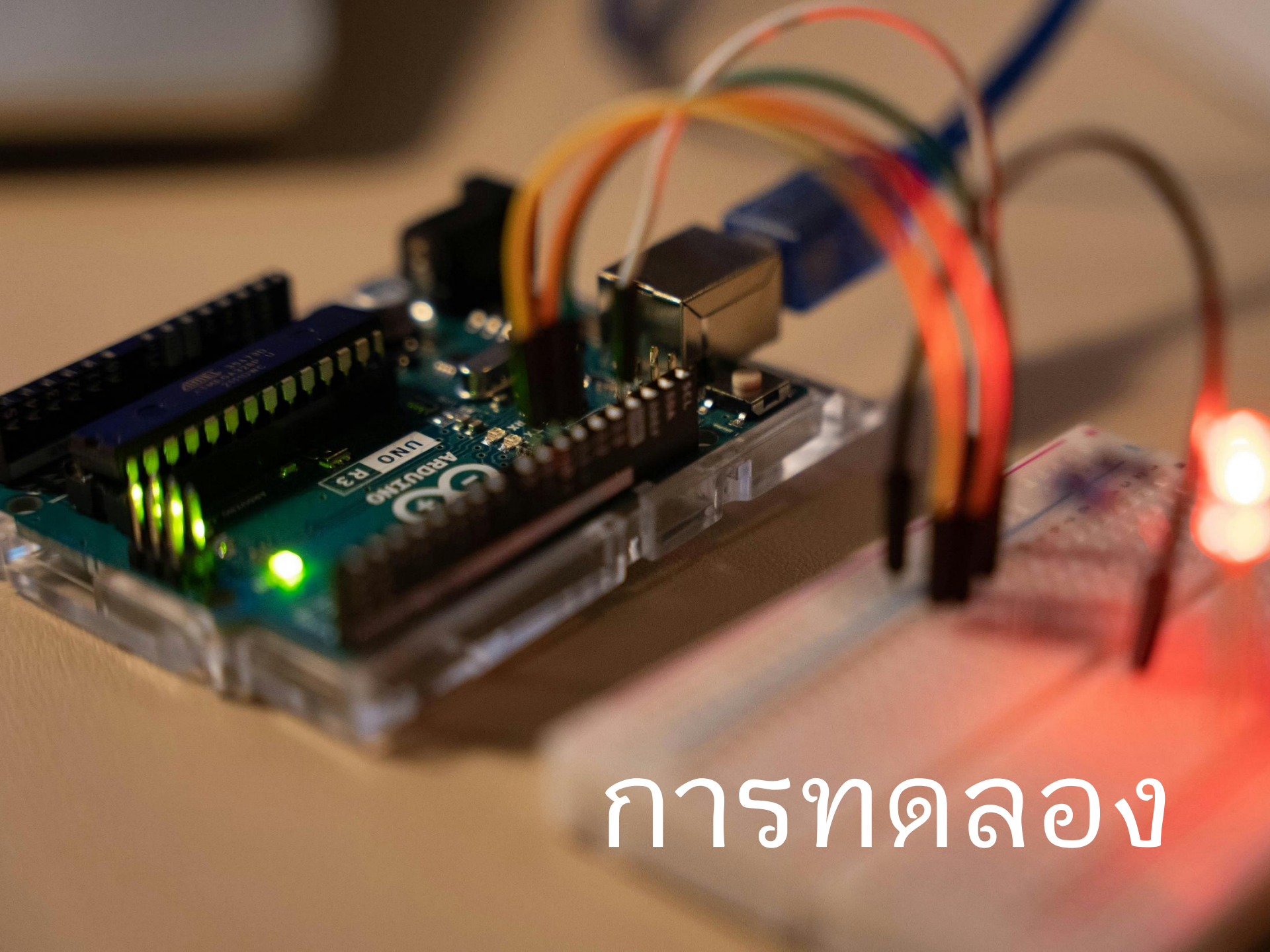
MQTT-Packet:
CONNACK

contains:	Example
sessionPresent	true
returnCode	0

MQTT-Packet:
DISCONNECT

no payload

Client ID จะต้องไม่ซ้ำกัน ถ้ามี ID ซ้ำกัน อุปกรณ์ที่อยู่ก่อน จะโดน Broker เตะออก

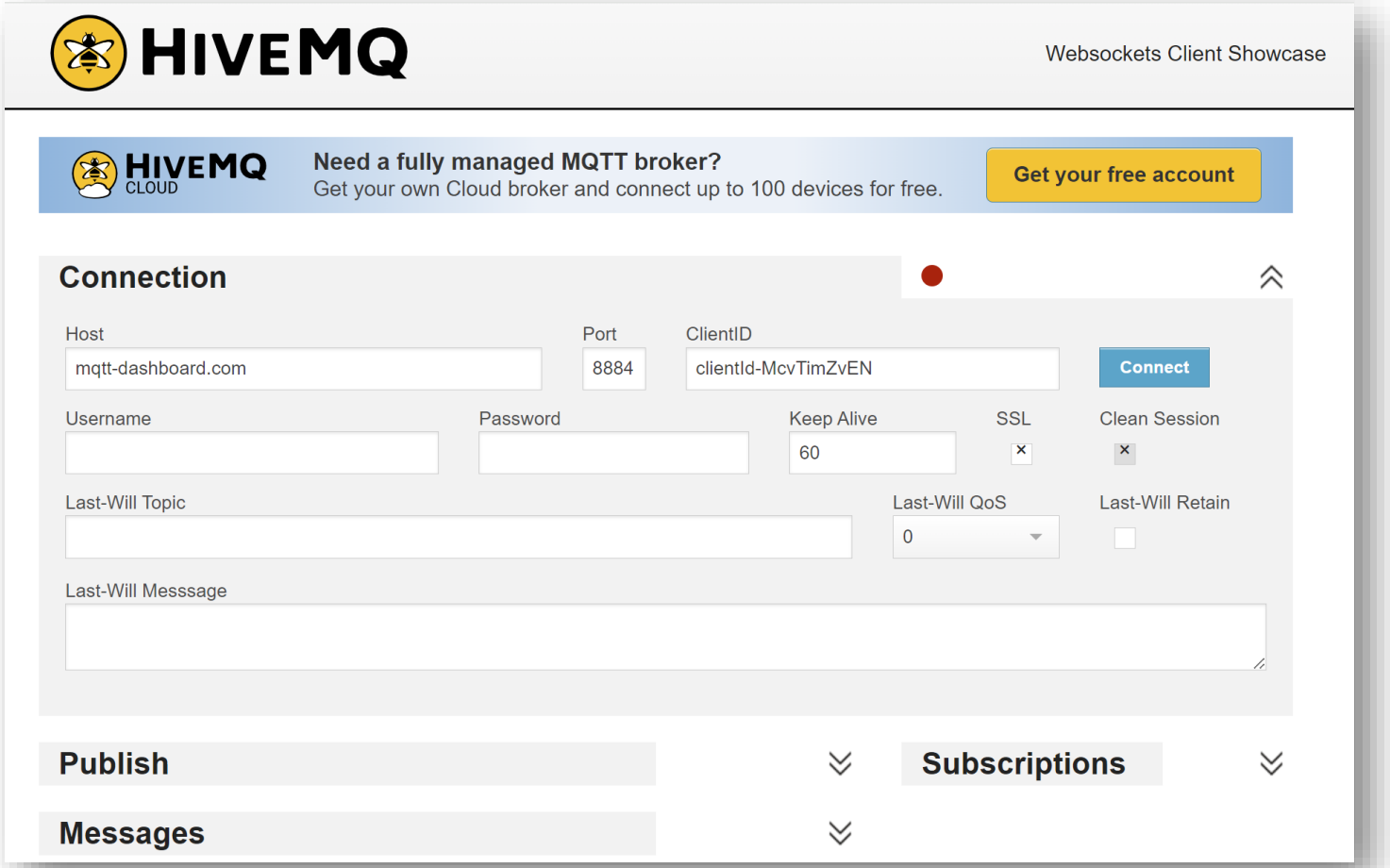


การทดลอง

การทดลองที่ 1. Public MQTT Broker

- เข้า Website ที่ URL

  http://www.hivemq.com/demos/websocket-client/



http://www.hivemq.com/demos/websocket-client/



การทดลองที่ 1. Public MQTT Broker

- Connect
  - กดปุ่ม Connect
- Subscribe
  - กด Add Topic Subscription เพื่อ add หัวข้อ ที่ต้องการ subscribe
  - กด Subscribe
- Publish
  - กำหนด topic ที่ต้องการจะ publish
  - พิมพ์ ข้อความ ที่จะ publish
  - กด publish

Feature ของ MQTT

1. Last Will and Testament

- Last Will แปลว่า พินัยกรรมฉบับสุดท้าย” หรือ “ความประสงค์ครั้งสุดท้าย
- testament แปลว่า “เป็นหลักฐาน/เป็นสิ่งยืนยันถึง...”

ดังนั้น

- Last Will and Testament แปลว่า ข้อความที่ฝากไว้ก่อนตาย/ก่อนขาดการติดต่อ หรือ ข้อความสุดท้ายที่ฝากไว้ หากหายไปโดยไม่ได้ร่ำลา
- LWT เป็น **Feature มาตรฐานของ MQTT**

Feature ของ MQTT

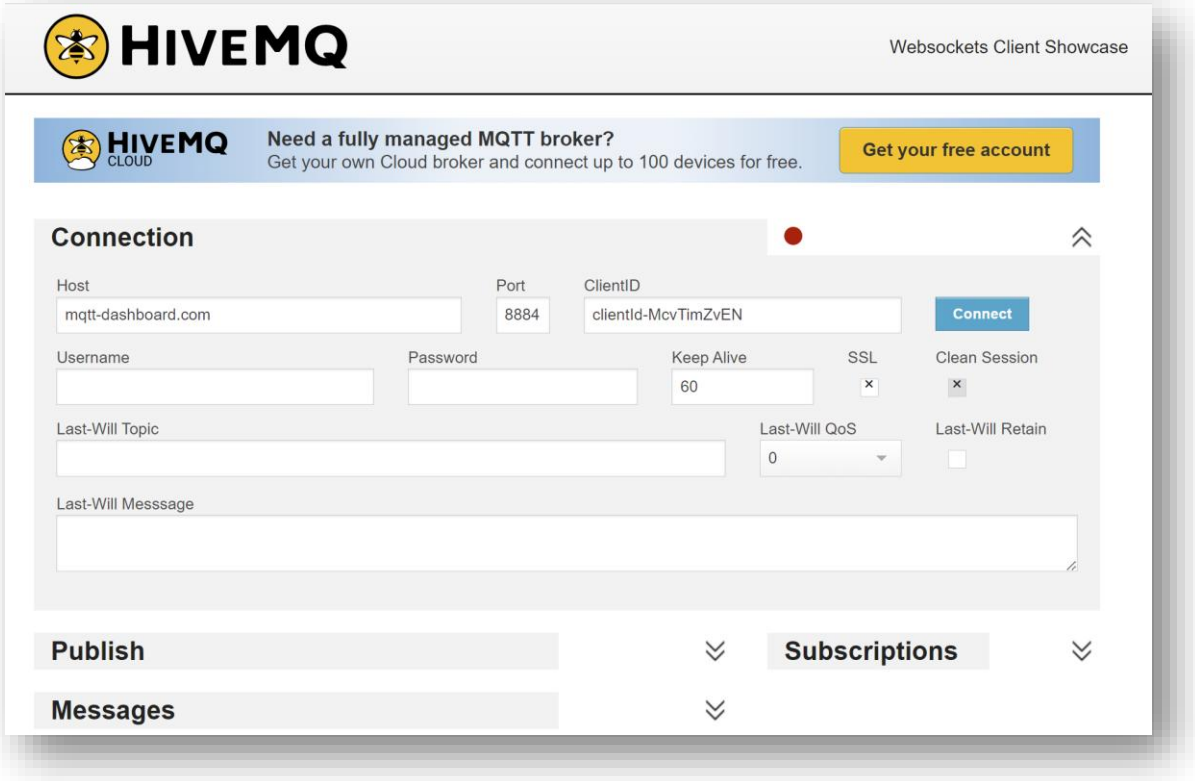
1. Last Will and Testament

- หลักการของ Last Will and Testament (LWT)
  - LWT คือ ข้อความที่ broker จะส่งออกอัตโนมัติ เมื่อ client หลุดการเชื่อมต่อแบบผิดปกติ เช่น ไฟดับ , network หลุด , โปรแกรม crash , ไม่ส่ง keep-alive ตามเวลาที่กำหนด
  - เมื่อเกิดเหตุการณ์ต่าง ๆ เหล่านี้ Broker จะทำการ publish ข้อความ LWT ไปยัง Topic ที่กำหนดไว้

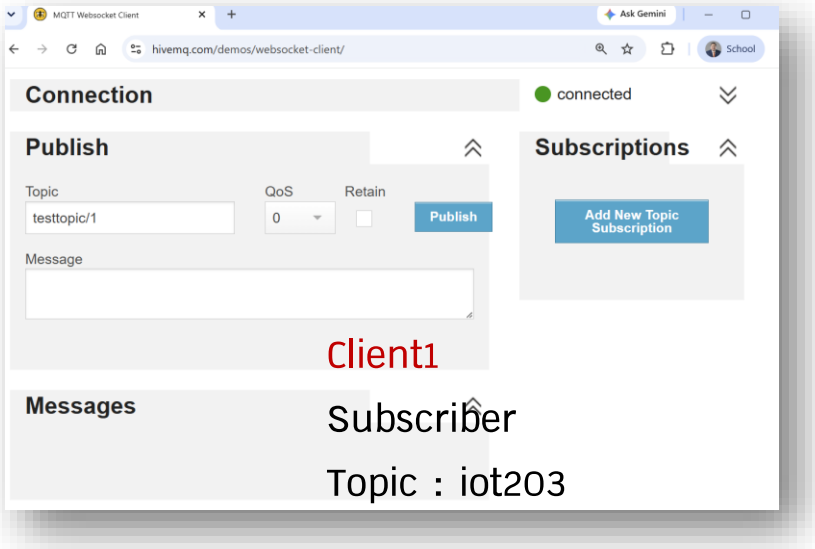
การทดลองที่ 1.1 Public MQTT Broker

- เปิด web browser เข้า Website ที่ URL
  http://www.hivemq.com/demos/websocket-client/

  จำนวน 3 หน้าจอ



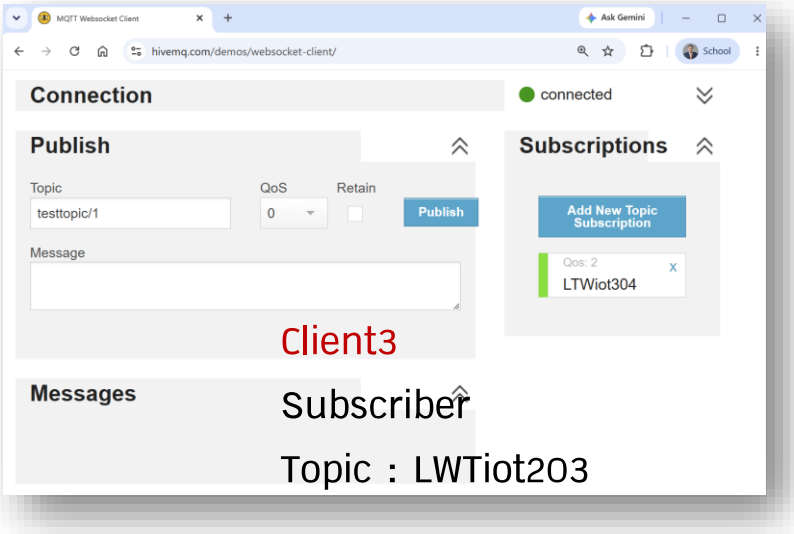
การทดลองที่ 1.1 Last Will and Testament



Client1

Subscriber

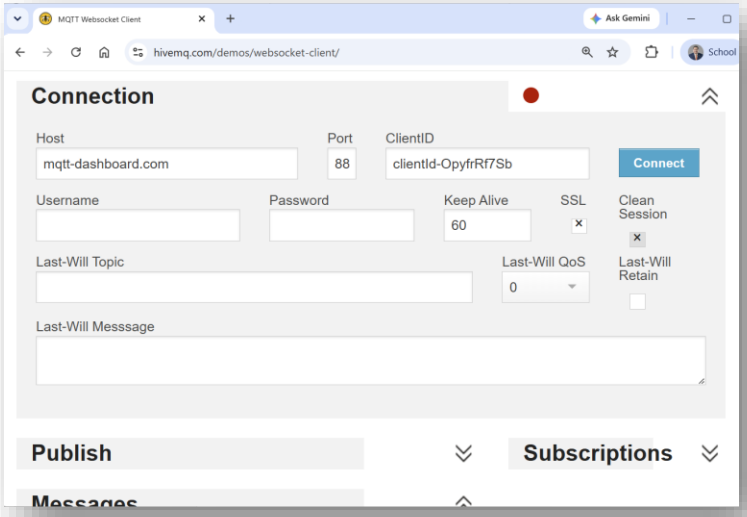
Topic : iot203



Client3

Subscriber

Topic : LWTiot203



Client2

Publisher

ตั้งค่า LWT-Topic : LTWiot203

ตั้งค่า LWT-Message : Alert!!!

ทำการ Publish ไปยัง iot203

จากนั้นทำการปิด Browser

การทดลองที่ 1.1 Last Will and Testament

- เปิด web browser จำนวน 3 หน้าจอ
  - Client 1 (Subscriber)
    - ทำการ Connect ไปยัง Broker
    - ทำการ Subscribe ไปยัง Topic ใดๆ ที่กำหนด เช่น Topic : IoT304
  - Client 2 (Publisher)
    - ตั้งค่า LWT (Last-Will Topic, Last-Will Message)
      - เช่น Last-Will Topic : LWTiot304 , Last-Will Message : Alert !!!
    - ทำการ Connect ไปยัง Broker
    - ทำการลอง Publish ไปยัง Topic ใดๆ ที่กำหนด เช่น Topic : IoT304
    - สังเกตว่า จอที่ 1 ได้รับข้อความ

การทดลองที่ 1.1 Last Will and Testament

- Client 3 (LWT Subscriber)
  - ทำการ Connect ไปยัง Broker
  - ทำการ Subscribe ไปยัง Topic ของ LWT
    - ในที่นี้ ทำการ subscribe Last-Will Topic : LTWiot304

- จากนั้นทำการ ปิด Browser ของ Client 2 หรือ Refresh (ห้ามกด Disconnect)
- ผลลัพธ์ที่ได้ : Client 3 จะได้รับข้อความอัตโนมัติว่า Alert!!!

Feature ของ MQTT

2. Session Management (Stateful Connection)

- MQTT สามารถรักษา “ Session ของ Client” ไว้ที่ Broker แม้ Client จะหลุดการเชื่อมต่อชั่วคราว
- ปัญหาของระบบ IoT คือ Client อาจ Offline ได้ตลอดเวลา เช่น

  Wi-Fi หลุด , Internet ขัดข้อง
- ถ้า Subscriber มีการเปิดใช้ Session เมื่อ Subscriber กลับมาเชื่อมต่อ ก็จะได้รับข้อมูลที่หายไป ในช่วงที่ขาดการเชื่อมต่อ
- Session Management ใช้ได้กับ MQTT Version 5

Feature ของ MQTT

2. Session Management (Stateful Connection)

- พารามิเตอร์หลักที่ต้องใช้ คือ
  - Client ID ต้องมีค่าเดิม เพื่อทราบว่า อุปกรณ์เดิมได้กลับมา
  - Clean Start ต้องมีค่าเป็น False (โดยค่า default จะมีค่าเป็น True ซึ่งหมายถึงให้เปิด Session ใหม่ ทุกครั้งที่มีการเชื่อมต่อ)
  - Session Expiry Interval คือ การกำหนดว่า Broker จะเก็บ Session ไว้นานเท่าไร หลังจาก Client Disconnect

Feature ของ MQTT

3. Keep Alive

- Keep Alive **คือ** ค่าที่กำหนดให้ client ต้องส่ง packet ไปยัง broker ภายในเวลาที่กำหนด (วินาที) เพื่อรักษา **MQTT connection** และตรวจสอบว่า connection ยัง active
- Keep Alive เหมือนเป็นการตรวจสอบว่าอีกฝ่ายยังออนไลน์อยู่หรือไม่ ?
- Client ต้องการบอก Broker ว่า ถ้าไม่มี MQTT Control Packet ใด ๆ ส่งมา ฉันจะส่ง PING Request เพื่อบอกว่า Client ยังมีชีวิตอยู่

- ถ้า broker ไม่ได้รับ packet ภายในเวลาที่กำหนด
  - 1.5 × Keep Alive
  - broker จะถือว่า Client **Disconnect** และอาจ trigger **Last Will and Testament (LWT)**

Feature ของ MQTT

3. Keep Alive

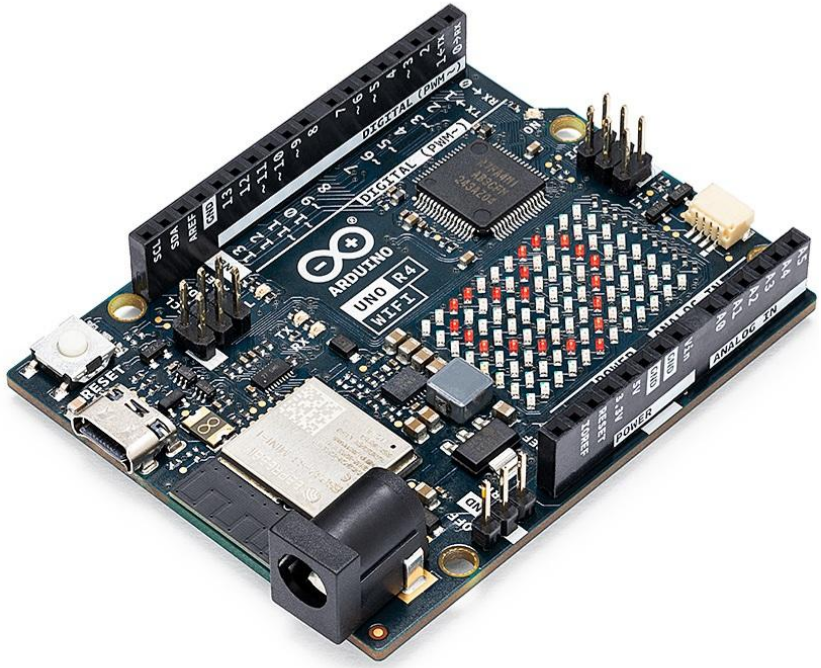
หลักการทำงานของ Keep Alive

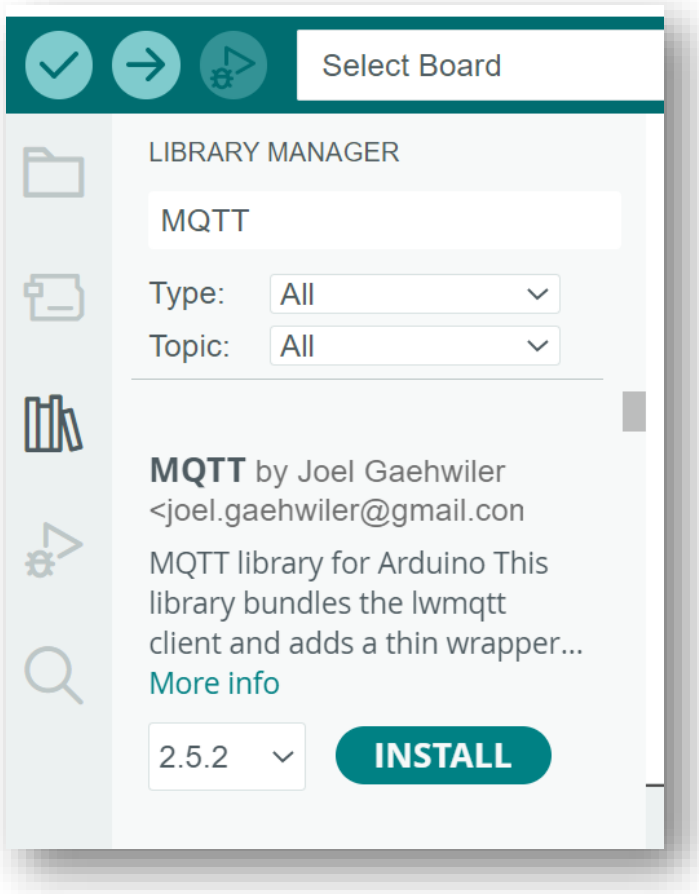
- เมื่อ Client เชื่อมต่อกับ broker
- Client กำหนดค่า **Keep Alive** เช่น Keep Alive = 60 seconds
- ถ้าในช่วง **60 วินาที** ไม่มีการส่ง packet ใด ๆ เช่น PUBLISH , SUBSCRIBE , PUBACK
  - Client จะส่ง packet “PINGREQ” ไปยัง broker
  - Broker จะตอบกลับ “PINGRESP” เพื่อยืนยันว่า connection ยังใช้งานได้

Laboratory : Library MQTT

ติดตั้ง Library

ค้นหา MQTT จากนั้นกด **INSTALL**





https://github.com/256dpi/arduino-mqtt

Library MQTT

- `MQTTClient client;` สร้าง object จาก class MQTTClient

  ค่า default buffer คือ 128 bytes

- `client.begin(MQTT_BROKER_ADRRESS, MQTT_PORT, network);`

  เริ่มต้นการเชื่อมต่อ (initialize) ระหว่าง MQTT client กับ MQTT broker โดยใช้ network client

- `client.onMessage(messageReceived);`

  คือการ **ตั้ง Callback Function** เมื่อมีข้อความ (message) เข้ามาจาก MQTT Broker — ให้เรียกฟังก์ชัน messageReceived เพื่อจัดการข้อความนั้น

- `client.connect(clientID, username, password);`

  ให้ client พยายามเชื่อมต่อกับ MQTT Broker โดยใช้ชื่อ clientID, ชื่อผู้ใช้ (username), และรหัสผ่าน (password)

Library MQTT

- `client.connected()` ตรวจสอบการเชื่อมต่อ เชื่อมต่อ : true, หลุด : False
- `client.subscribe("some/topic", qos);`
- `client.unsubscribe("some/topic");`
- `client.publish("some/topic", payload, retained, qos);`
- `client.loop();`
- `client.disconnect();`

Wi-Fi®

The ESP32 onboard the UNO R4 WiFi is used to give the board Wi-Fi® capabilities. The Wi-Fi® module has a bitrate of up to 150 Mbps. The ESP32 module has a built in trace-antenna, meaning that you do not need an external one to use the connectivity features of the board. However, this trace antenna is shared with the Bluetooth® module, which means that you cannot use Bluetooth® and Wi-Fi® at the same time.

To use the Wi-Fi® features of the UNO R4 WiFi, use the **WiFiS3** library that is built in to the UNO R4 Board Package.

Wi-Fi®

- Wi-Fi® support with 802.11 b/g/n standard (Wi-Fi® 4)
- Bit rate at up to 150 Mbps
- 2.4 GHz band

https://docs.arduino.cc/tutorials/uno-r4-wifi/wifi-examples/

Library WiFiS3

เป็น ไลบรารีสำหรับควบคุมการเชื่อมต่อ WiFi บนบอร์ด Arduino

- `WiFiClient network;` สร้าง object จาก class WiFiClient

การเชื่อมต่อ WiFi

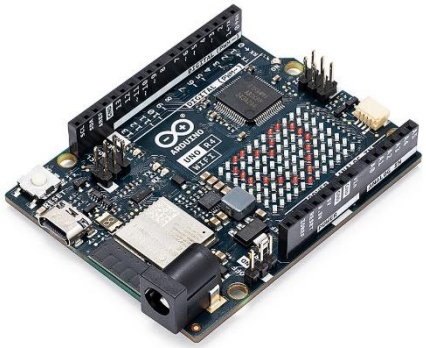
- `WiFi.begin(ssid, pass);` เริ่มต้นการเชื่อมต่อ WiFi
- `WiFi.disconnect()`
- `WiFi.status()` ตรวจสอบสถานะการเชื่อมต่อ ผลลัพธ์ เช่น WL_CONNECTED , WL_IDLE_STATUS , WL_NO_SSID_AVAIL
- `WiFi.reconnect()`

การจัดการข้อมูล

- `WiFi.localIP()` ค่า IP ที่บอร์ดได้รับจาก router
- `WiFi.gatewayIP()` ค่า IP ของ gateway (router)
- `WiFi.SSID()` ชื่อ Wi-Fi ที่กำลังเชื่อมอยู่
- `WiFi.RSSI()` ค่าความแรงสัญญาณ (RSSI) หน่วย dBm

การทดลองที่ 2. MQTT with UNO R4 - WiFi

- นำมือถือของ นศ เปิด Hot Spot ตั้งค่า SSID, Password
- แก้ค่าต่างๆ ในโปรแกรม ให้ถูกต้อง เช่น SSID, Password ,ClientID , Broker Address , Topic
- ทำการ Upload Code ที่แก้ค่าแล้ว ลงบนบอร์ด
- เปิด Serial Monitor เพื่อดูค่า
- เปิด Website hiveMQ ใส่ค่า Topic ที่ publish, Subscribe ให้ถูกต้อง
- สังเกตค่าที่ส่งมา ลองทำการ Pub/Sub ไปยังบอร์ด





HIVEMQ

Code ตัวอย่าง Publish/Subscribe MQTT

- https://github.com/ObeOne-KMITL/lab_iot/blob/main/arduino_R4/mqtt.ino

แก้ Code ในส่วนนี้

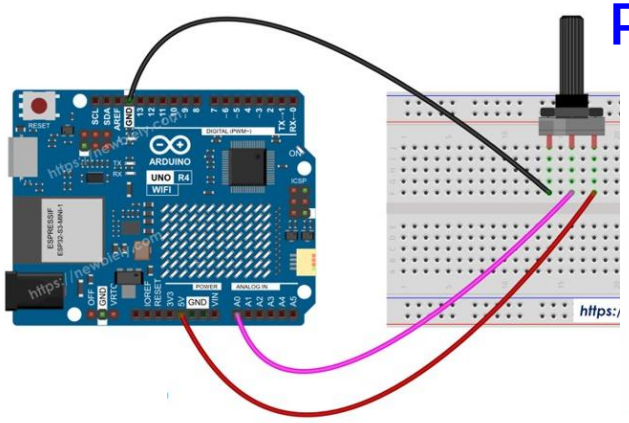
```
const char WIFI_SSID[] = "kmitl-wifi";     // CHANGE TO YOUR WIFI SSID
const char WIFI_PASSWORD[] = "kmitl";  // CHANGE TO YOUR WIFI PASSWORD

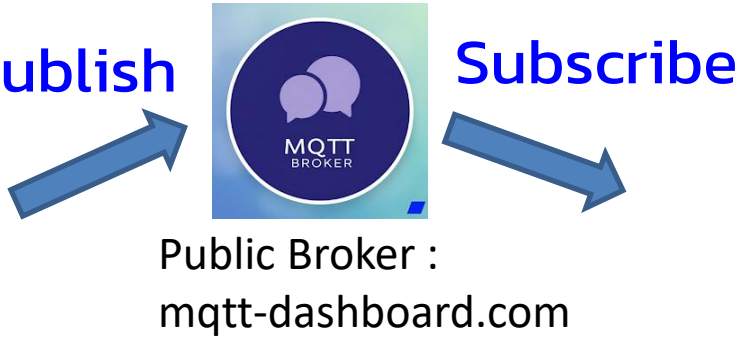
const char MQTT_BROKER_ADRRESS[] = "mqtt-dashboard.com";  // CHANGE TO MQTT BROKER'S ADDRESS
//const char MQTT_BROKER_ADRRESS[] = "192.168.0.11";  // CHANGE TO MQTT BROKER'S IP ADDRESS
const int MQTT_PORT = 1883;
const char MQTT_CLIENT_ID[] = "arduino-uno-r4-panwit";  // CHANGE IT AS YOU DESIRE
const char MQTT_USERNAME[] = "";               // CHANGE IT IF REQUIRED, empty if not required
const char MQTT_PASSWORD[] = "";               // CHANGE IT IF REQUIRED, empty if not required

// The MQTT topics that Arduino should publish/subscribe
const char PUBLISH_TOPIC[] = "arduino-uno-r4-panwit/send";       // CHANGE IT AS YOU DESIRE
const char SUBSCRIBE_TOPIC[] = "arduino-uno-r4-panwit/receive";  // CHANGE IT AS YOU DESIRE
```

การทดลองที่ 3. Board : Publish -> Web : Subscribe

AnalogRead ต่อตัวต้านทานปรับค่าได้ โดยขากลางต่อเข้าที่ขา A0



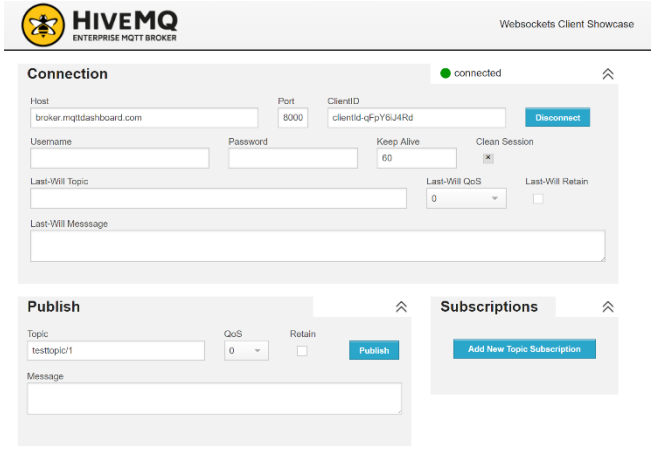


Public Broker :
mqtt-dashboard.com

Web Browser

ฝั่งรับ (Subscriber)
เปิด website
http://www.hivemq.com/demos/websocket-client/

ทำการเชื่อมต่อ , ทำการ Subscribe และสังเกตค่าที่ได้

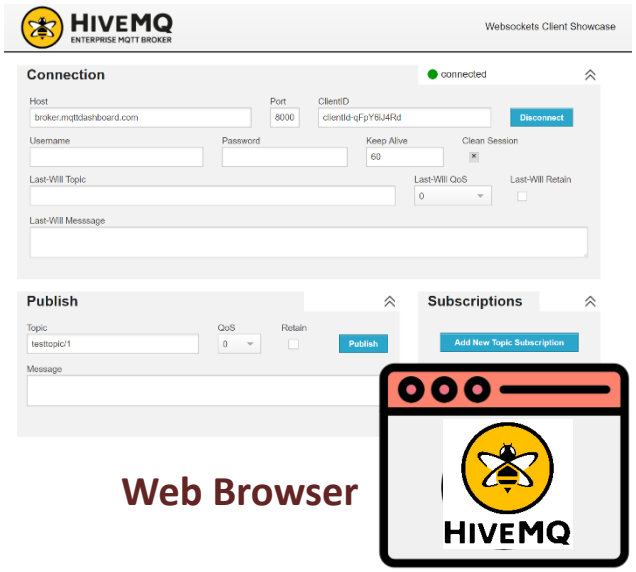


Note!!!
ฝั่งส่ง (Publisher)
ส่วนของโปรแกรมที่ใช้ส่งคือ

```
int  val = analogRead(A0);
String val_str = String(val);
char messageBuffer[10];
val_str.toCharArray(messageBuffer, 10);
mqtt.publish(PUBLISH_TOPIC, messageBuffer);
```

หลักการคือ แปลค่า Interger ให้เป็น String ก่อน
จากนั้นเปลี่ยนจาก String ให้เป็น Array

การทดลองที่ 4. Web : Publish -> Board : Subscribe



Web Browser

Publish

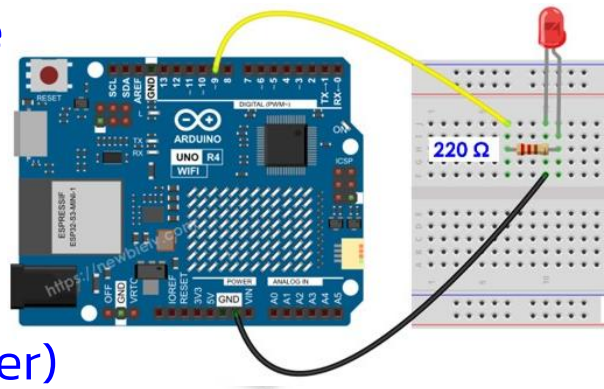
Public Broker :
mqtt-dashboard.com

MQTT BROKER

Subscribe

AnalogWrite

ต่อตัวต้านทาน + LED เข้าขา 9



ฝั่งส่ง (Publisher)

ให้ publish ค่า 0-255
แล้วไปสั่งให้ หลอด LED
หรี่ สว่าง ตามค่าที่ส่งมา

0 คือ ดับ
255 คือ สว่างที่สุด

ฝั่งรับ (Subscriber)

ในฟังก์ชั่น messageReceived

```
void messageReceived (String &topic,
String &payload)
```

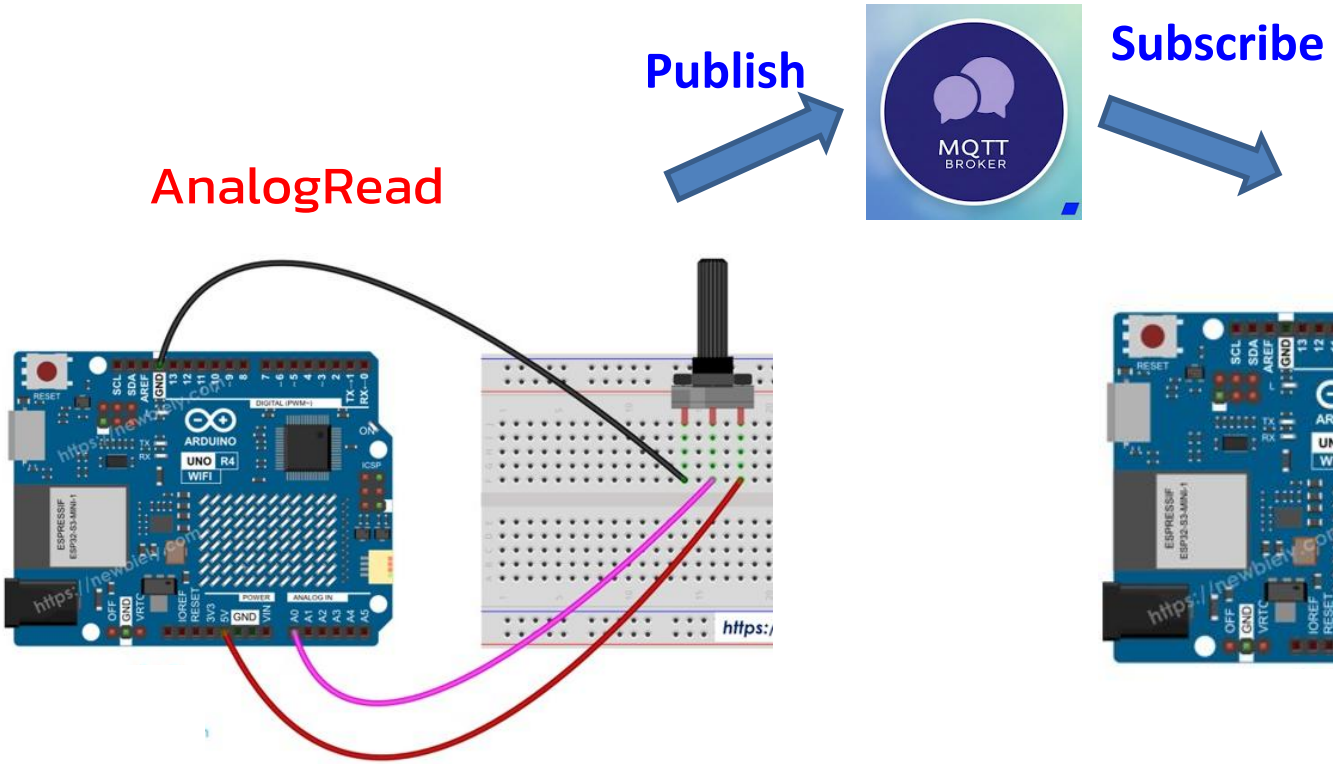
เพิ่ม Code โดยจะต้องเปลี่ยน payload จาก String ให้เป็น Interger ก่อน

```
val = payload.toInt();
analogWrite(9,val);
```

การทดลองที่ 5. สั่งหรี่ไฟผ่าน Internet

ใช้ 2 board

Public Broker :
mqtt-dashboard.com



การทดลองอื่นๆ

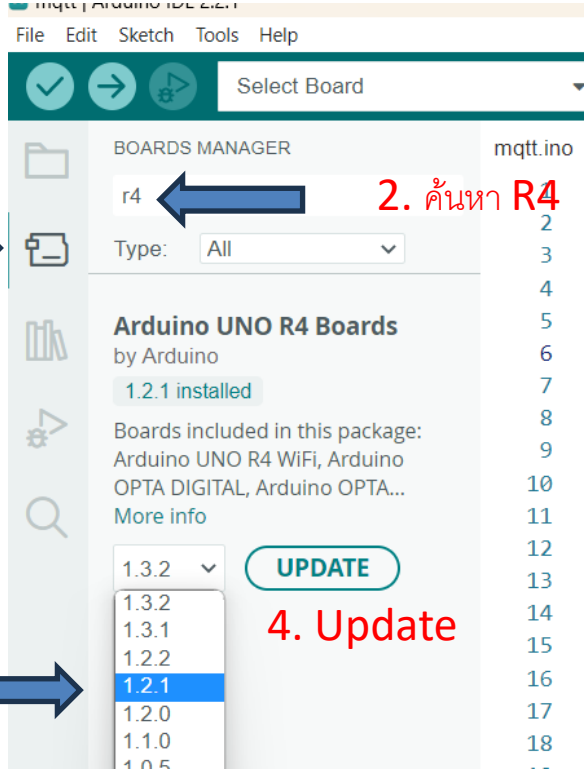
- ใช้ 2 board
  - คนส่ง (Publisher) ทำการ publish ส่งค่าอุณหภูมิ
  - คนรับ (Subscriber) นำค่า อุณหภูมิที่ได้รับไปแสดงออกจอ LCD หรือ ออก RGB Color LED ที่มีสีตามอุณหภูมิ ร้อน : แดง เย็น : เขียว

- ใช้ 2 board
  - คนส่ง (Publisher) ทำการ publish ค่าจาก Switch 3 ตัว
  - คนรับ (Subscriber) นำค่าที่ได้จากไปสั่ง ปิด-เปิด RGB LED
    - เมื่อรับค่าจาก Switch 1 ให้แสดง RED
    - เมื่อรับค่าจาก Switch 2 ให้แสดง GREEN
    - เมื่อรับค่าจาก Switch 3 ให้แสดง BLUE

กรณีเกิด bug

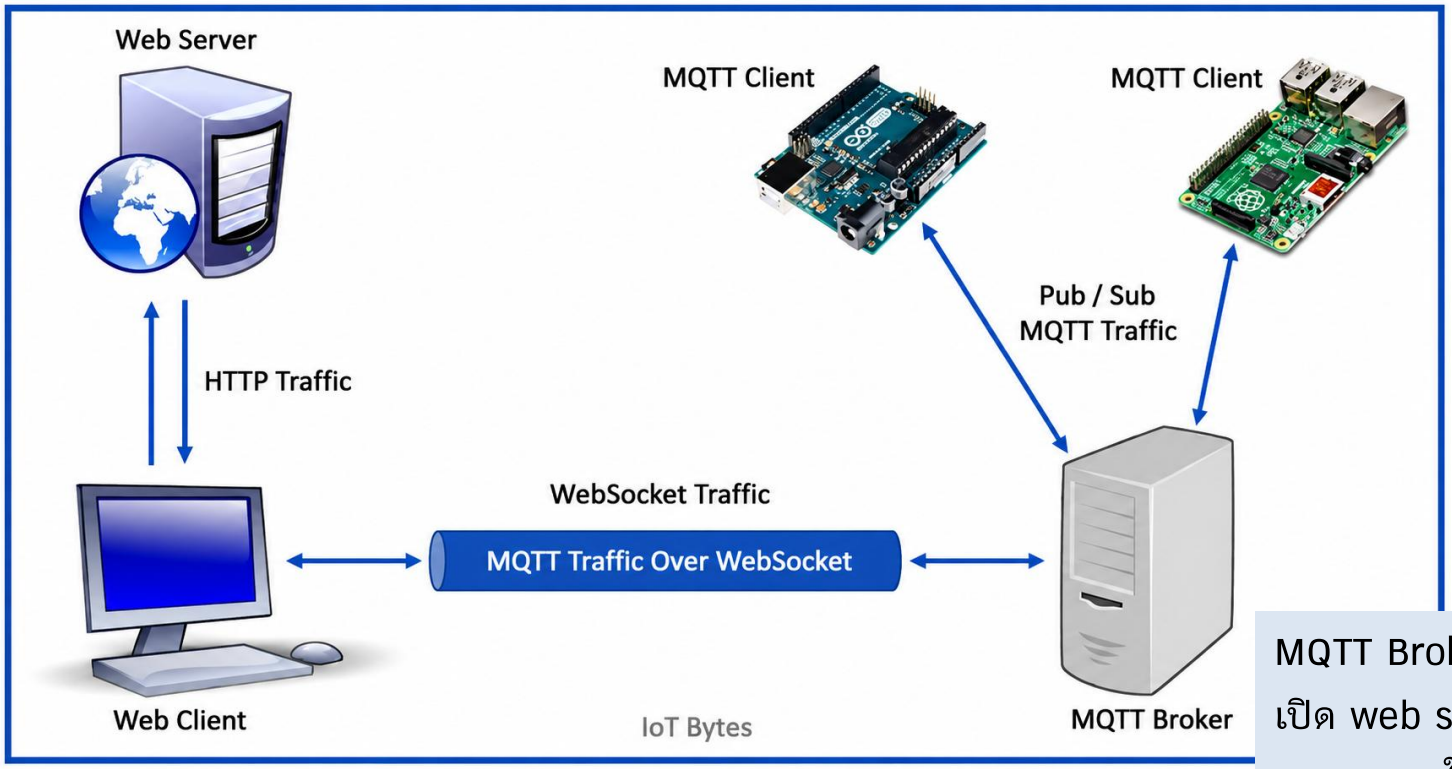
- Code ตัวอย่างที่ให้ไป ถูกต้องแล้ว
- หลายคนทดลองแล้ว มันต่อ broker ไม่ติด มันขึ้น .....
- ให้แก้ Version ของ Board manager ให้เป็นตามรูป

1. กด board manager



WEB Socket

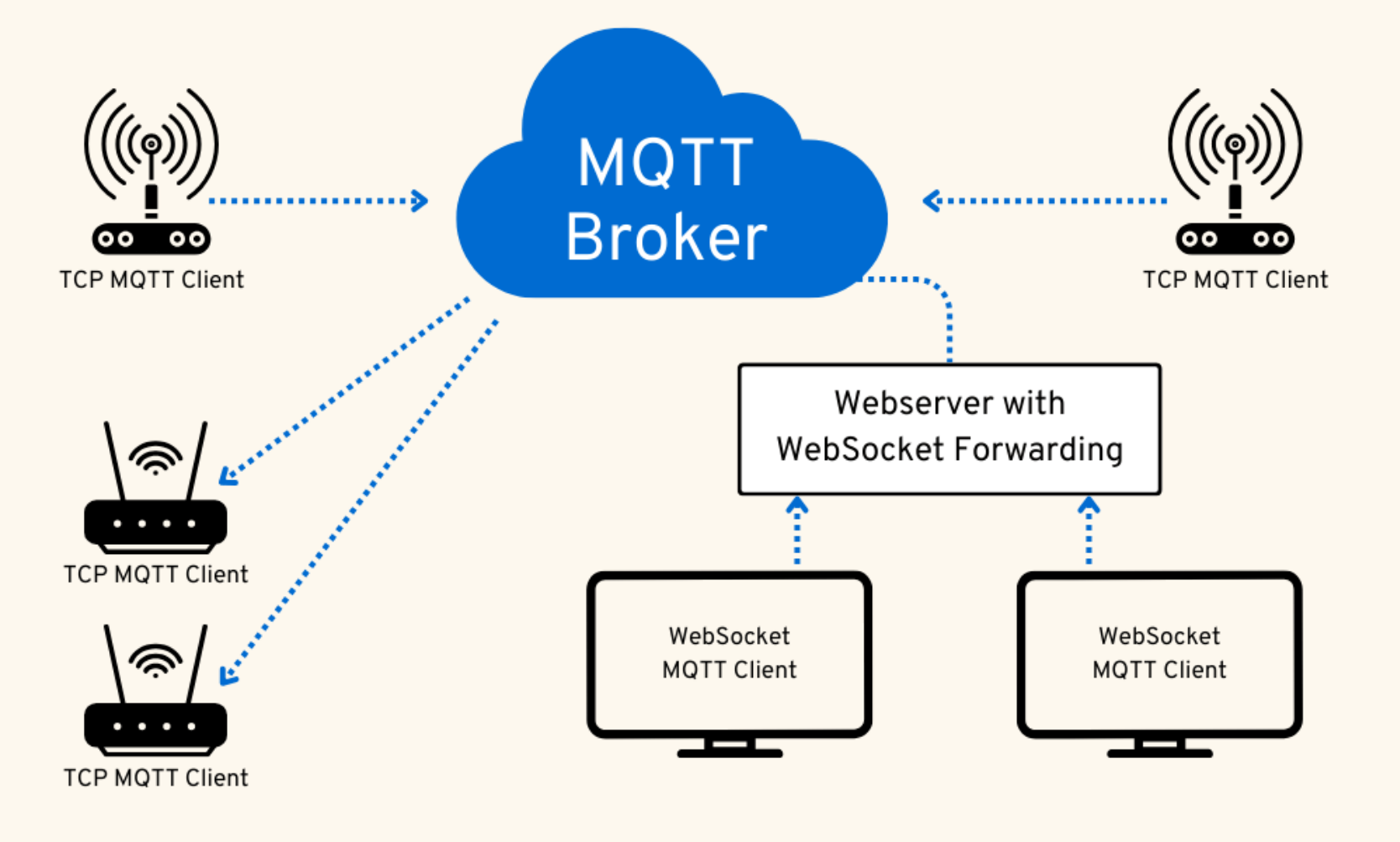
WebSocket คือ โปรโตคอลที่ใช้ในการสื่อสารแบบสองทาง (bidirectional) เปิดการเชื่อมต่อค้างอยู่ตลอดเวลา ทำให้สามารถส่งข้อมูลได้ real-time และลดความล่าช้าในการติดต่อสื่อสาร



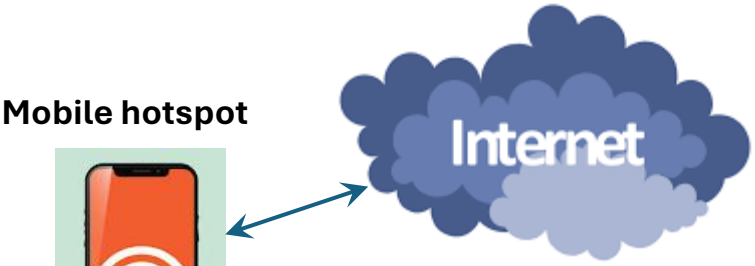
MQTT Broker ต้องเปิด web socket จึงจะสามารถใช้งานได้

MQTT over Websockets

MQTT over Websockets



Lab : Network Topology





mqtt-dashboard.com

หรือ

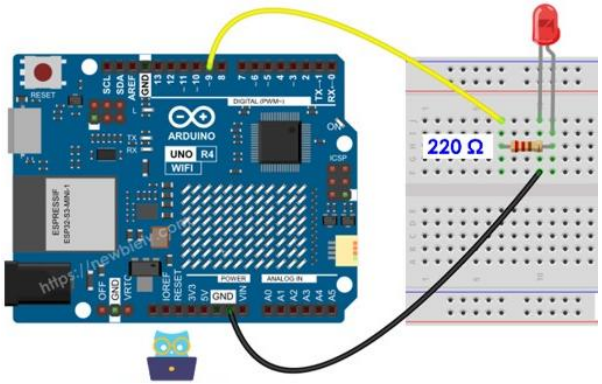
phycom.it.kmitl.ac.th

Port

mqtt : 1883

web socket : 8884





เปิด web browser

Run ไฟล์ control_rev1.html

ไฟล์ control_rev1.html แก้ broker, port, topic

Arduino Subscriber

ต่อ ตัวต้านทาน 220 โอห์ม

และต่อหลอด LED และต่อ WiFi

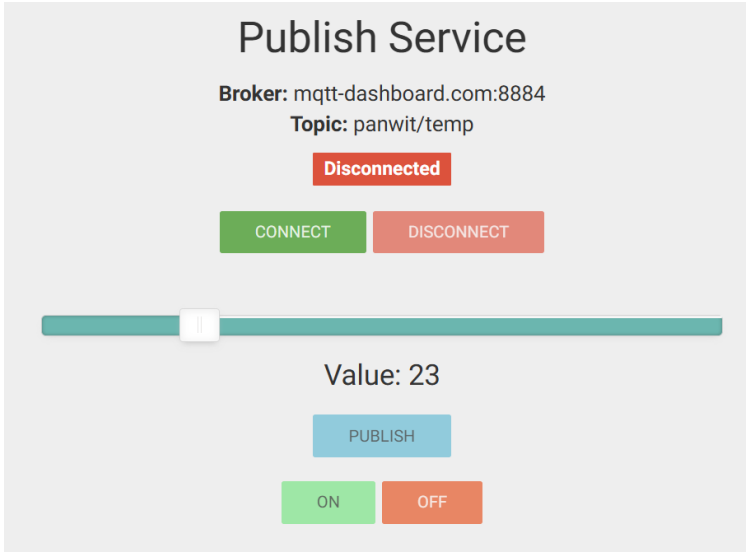
การทดลองที่ 6. MQTT Control via Websocket

1) เตรียมไฟล์

- แตกไฟล์ web.rar (Unzip) ไฟล์ทั้งหมดไว้ใน Folder เดียวกัน
- เปิดไฟล์ control_rev1.html

2) การควบคุมผ่าน MQTT

- ไฟล์ตัวอย่างทำหน้าที่ Publish ข้อมูลไปยัง Topic ที่กำหนด
  - เมื่อกดปุ่ม ON จะ publish ค่า on ไปยัง Broker
  - เมื่อกดปุ่ม OFF จะ publish ค่า off ไปยัง Broker
  - เมื่อ Slide bar จะส่งค่าตามที่ Slide



การทดลองที่ 6. MQTT Control via Websocket

1. แก้ไขไฟล์ control_rev1.html ดังนี้

```
var MQTTbroker = 'mqtt-dashboard.com';   // broker
var MQTTport = 8884;                     // port ของ Webseocket ที่เปิดไว้
var MQTTsubTopic = 'temp';               //แก้ชื่อ topic ได้ตามใจชอบ
```

2. ทำการ Upload ไฟล์ ไปยัง web hosting (หรือ run ที่ client)

3. เขียนโปรแกรม ที่ Arduino เพื่อ Subscribe ค่าที่ส่งมาจากไฟล์ control_rev1.html
   - เมื่อกดปุ่ม ON ให้ LED ติด เมื่อกดปุ่ม OFF ให้ LED ดับ
   - เมื่อเลื่อน Slide bar ให้ LED หรี่สว่างตามค่าที่ Publish มา

แบบฝึกหัดที่ 1. MQTT Control via Websocket

1. แก้ไขไฟล์ control-rev1.html ให้

   ควบคุมการผสมสี Color LED เช่น

   1. Red : ON , Green : OFF , Blue : ON ให้ Color LED แสดงสีม่วง




THE END